

ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE



ROBOTIQUE, 2025-26

Master thesis

DASH-01 :
RL Framework to Develop a Bipedal
Humanoid Robot Aimed at Breaking the
Speed Record

REHAssist

13.04.2025 - 04.09.2025

Author:

Nathann MORAND
Robotics M.Sc.

Supervisors:

Dr. Mohamed BOURI

Abstract

This thesis investigates the use of reinforcement learning to develop a high-speed running controller for DASH-01, a 15 kg bipedal robot designed for high-speed locomotion. The objective was to determine how far reinforcement learning could push the running speed of a small, lightweight biped while accounting for the limitations of the physical platform.

The work follows the complete path from the robot model to a deployable controller. The robot was reconstructed in MuJoCo from the mechanical design, and the actuator, sensing and communication interfaces were characterised experimentally. These measurements were used to correct the simulation and to establish a first-principles upper bound on the achievable running speed. Under the measured kinematic, inertial and actuator constraints, this bound was estimated at approximately 6 m/s.

Several reinforcement-learning formulations were then investigated. Direct torque control did not converge within the available training budget and was initially rejected because of an assumed communication-bandwidth limitation. Per-step position control converged to several non-locomotive solutions, including skating. The action space was consequently structured around a periodic gait generator. Fourier-series and explicit oscillator representations were compared, together with impedance parameters and fast balance reflexes. A curriculum progressively released the robot's base degrees of freedom, allowing locomotion to be learned before full-body balance was required.

The experiments show that the formulation of the learning problem was at least as important as the choice of learning algorithm. In particular, reward maximisation could produce apparently successful policies that exploited the simulation rather than running, while external balance assistance could produce policies dependent on the assistance. The final curriculum reached sustained running in simulation, with the best policies reaching approximately 3 m/s. The main remaining limitation was the transition to a fully free pitch and roll plant, where the learned controllers encountered a stability wall associated with both the policy and the physical geometry of the leg.

The resulting framework provides a measured and deployable RL pipeline for high-speed bipedal locomotion, together with an experimentally grounded estimate of the remaining gap between the current robot and its physical speed limit. The results indicate that further progress is likely to require changes to the mechanical plant and actuator capability rather than simply increasing the reinforcement-learning budget.

Table of Contents

Abstract	2
1 Introduction and context	5
1.1 The platform and the objective	5
1.2 The objective moved from record pace to a measured demonstrator	5
1.3 Contributions	5
1.4 Reading map	6
2 Background and literature review	7
2.1 Definitions	7
2.2 Abbreviations	7
2.3 Bipedal locomotion approach	7
2.4 Reinforcement learning for locomotion	9
2.5 Platforms, and where DASH-01 sits	11
2.6 The gap	12
3 The robot and its digital twin	13
3.1 DASH-01	13
3.2 Selecting hardware	14
3.3 The digital twin	15
3.4 Digital twin validation and plant identification	16
3.5 Deployment stack	20
3.6 Running speed ceiling estimation	23
4 Reinforcement learning: attempts and architecture	26
4.1 Formulation	26
4.2 The toy problem	26
4.3 Observation space	27
4.4 Reward design	27
4.5 Evaluation protocol	27
4.6 The action-space ladder	28
4.7 Curriculum design	30
4.8 The pitch wall	31
5 Results	33
5.1 Hardware delivered	33
5.2 Running speed ceiling estimation	33
5.3 The runner	33
5.4 Legality of the gait against the measured envelope	34
5.5 Robustness margins	34
5.6 Deployment readiness	34
6 Discussion	36
6.1 Method-level lessons	36
6.2 Mechanical limits and performance levers	36
7 Conclusion and future work	38
7.1 What was delivered	38
7.2 Limitations	38

7.3	Future work	39
8	Appendix	40
8.1	From CAD to a simulable model	40
8.2	Component databases	40
8.3	Logbook	41
	References	43

1 Introduction and context

This chapter states the objective, the platform, and the scope. It closes with the contributions and a reading map.

1.1 The platform and the objective

DASH-01 is a bipedal robot built at REHAssist. It carries two legs, six quasi-direct-drive actuators, a parallel 4-bar knee, a passive ankle spring and a single-board computer. The hardware budget was roughly CHF 5000. Figure 3.1 shows the machine as modelled.

The objective at kickoff was maximum running speed, with the bipedal 100m record as the ambition. That record has moved several times during the project. It now stands at 8.86s, set by Tiangong Ultra in Beijing on August 25, 2026 [2]. That is an average of about 40.6 km/h, faster than Usain Bolt’s 9.58s. Before this thesis started, the record was Cassie’s 24.73s over the same distance, set in 2022 [5].

The project inherited a partially finished robot. The work reported here therefore starts at modelling, and at the selection of the non-mechanical subsystems. It then runs through characterisation and the training of a policy.

1.2 The objective moved from record pace to a measured demonstrator

The record moved faster than the thesis. New bipedal speed marks now appear at roughly a monthly cadence, from teams with institutional budgets.

The measured plant does not admit record pace. Section 3.6 bounds this mechanism at about 6 m/s. The bound comes from first principles, by using the workspace of the leg and the impulse it can deliver.

The objective of this thesis was therefore restated as a measured demonstrator plus a reusable method. The deliverable is the chain from CAD to a trained policy.

The problem this addresses. Reinforcement learning has produced legged-locomotion results on research platforms. Reproducing them on a new, custom, imperfectly known robot is non-trivial. The plant parameters are uncertain. The actuators hide undocumented internal control loops. The sensors are commodity parts. The gap between “reinforcement learning walks in simulation” and “reinforcement learning walks on this robot” is dominated by modelling fidelity, actuator bandwidth and calibration.

Sensing scope. Proprioception and one base inertial measurement unit are the only sensors. Foot impact sensors were dropped due to lack of time rather than on evidence. The hardware provision for them remains in place, as four analogue inputs on the sensor hat. The literature prior that foot feedback helps convergence is therefore untested here.

1.3 Contributions

- **A CAD-to-MJCF pipeline**, with the loop-closure treatment URDF cannot express. The model was then validated against the backdriven workspace (Section 3.3).
- **A measured plant** (Section 3.4). It covers weighed segment masses, a torque constant identified by a known-mass method that breaks the degeneracy between K_t and gravity, the

Bode response of the drive’s internal position loop, inertial-sensor noise characteristics, loop timing and the CAN protocol.

- **A first-principles capability analysis** of running speed on that plant, validated against MuJoCo (Section 3.6).
- **An action-space and curriculum design** for the learned controller. Multiple action spaces were tested, and a curriculum was built to converge to a running policy on the measured plant (Sections 4.6.4 and 4.7).
- **A deployment stack on the robot.** It is a 200 Hz daemon on stock Linux, a web bring-up instrument that was used to perform the measurement of the robot characteristics. The deployment stack also includes safety mechanisms to prevent the robot from exiting the safe operating envelope (Section 3.5).
- **Two component databases**, covering 140 commercial actuators and roughly 430 battery cells. They were used to validate the choice of the actuators and the battery pack (Section 3.2, Section 8.2).

1.4 Reading map

Section 2 places the control approach among the published families and names the gap this work sits in. Section 3 finishes the robot, builds its digital twin, and replaces the assumed plant parameters with measurements. It closes with the running speed the measured plant admits. Section 4 develops the learned controller on that plant, one action space at a time. Section 5 reports what the robot and the policies deliver. Section 6 ranks the changes that would close the remaining gap, and Section 7 concludes.

2 Background and literature review

This chapter provides a brief context for bipedal locomotion and, more specifically, for the Reinforcement Learning framework used in the literature to address this problem.

2.1 Definitions

First, a few definitions are provided, as the following terms will recur throughout this thesis.

Underactuation is the presence of more degrees of freedom than independently actuated degrees of freedom.

Unilateral contact means that a foot can push against the ground but cannot pull it, and that the resulting contact force must lie within the **friction cone**.

The **sim-to-real gap** is the performance mismatch observed when transferring a policy from a simulated plant to the physical robot. This mismatch generally arises because the real plant lies outside the distribution of plants and disturbances encountered during training.

In the context of this thesis, **failure** refers to the termination of an episode following body contact with the ground or leaving the measured workspace.

Stability margin, in the context of Reinforcement Learning, is not necessarily expressed as a classical analytical stability margin. In this thesis, robustness is instead assessed through quantities such as the number of surviving episodes and the variance across random seeds.

Plant, borrowed from control theory, refers to the physical system to be controlled, in this case the robot.

2.2 Abbreviations

RL : Reinforcement Learning

CoM : Centre of Mass

2.3 Bipedal locomotion approach

Bipedal locomotion can be approached in different ways, depending in particular on the speed of motion and on the assumptions made about contact and dynamics.

The simplest case is a robot with flat feet walking on flat terrain at sufficiently low speed that inertial effects can be neglected. This was the approach used for WABOT-1 in 1973 [26].

In this case, maintaining stability can be reduced to keeping the projection of the CoM inside the support polygon formed by the feet.

As the robot moves faster, inertial effects can no longer be neglected. The relevant quantity is then not simply the projection of the CoM, but the Zero Moment Point (ZMP), which must remain within the support polygon. This approach was used, for example, in ASIMO [27].

Running introduces a flight phase, which is incompatible with the previous approaches that require at least one foot to remain in contact with the ground at all times.

To enable running, Raibert [20] proposed a foot-placement method based on the following relation:

$$x_f = \frac{1}{2} \cdot \dot{x} \cdot T_s + k \cdot (\dot{x} - \dot{x}_d) \quad (2.1)$$

where x_f is the fore-aft foot placement, measured from the hip at touchdown, in metres; $\dot{x}$ is the current forward speed in metres per second; $\dot{x}_d$ is the commanded forward speed, in metres per second; T_s is the stance duration, in seconds; and k is a velocity feedback gain, also expressed in seconds.

The first term corresponds to the neutral point, which produces no net fore-aft impulse. The second term provides velocity feedback: when the robot is moving faster than commanded, the foot is placed further forward, producing a decelerating effect during stance.

This lineage runs through systems such as BigDog [19] and later Atlas.

The underlying idea is that the gait can be stabilised, to a significant extent, by placing the feet at the appropriate location and time.

This also makes it possible to build running robots with point-contact feet, rather than requiring the large flat feet used by earlier approaches.

The approach was subsequently refined by solving the foot-placement and whole-body motion problem online, leading to what is commonly formulated as a Model Predictive Control approach [11].

This approach was central to the development of the early dynamic legged robots developed by Boston Dynamics.

These approaches rely on a sufficiently accurate model of the plant.

In practice, however, the plant parameters are only partially known and may vary with configuration, load, contact conditions, and hardware.

Classical model-based controllers are therefore designed around a particular model and possess a finite robustness margin with respect to modelling errors and disturbances. Beyond this margin, the controller may no longer be able to stabilise the system.

Reinforcement Learning provides an alternative way of obtaining this robustness. Rather than explicitly deriving a controller from a single plant model, RL trains a policy over a distribution of simulated plants. The policy can therefore be exposed to variations in plant parameters during training, for example through domain randomisation, with the objective of improving its robustness to model mismatch.

The problem is further complicated by the unilateral and saturating nature of ground contact. A foot can only push against the ground, not pull it, and the tangential component of the contact force is bounded by the friction cone. Exceeding this bound results in slipping.

A second complication is the timescale of the dynamics. A legged robot can, to a first approximation, be considered as an inverted pendulum with characteristic length h .

In the absence of corrective input, the characteristic timescale of the inverted pendulum is on the order of

$$\tau \sim \sqrt{\frac{h}{g}}, \quad (2.2)$$

which corresponds to approximately 300 ms for a characteristic length of 1 m.

During running, however, contact with the ground may last only a few tens of milliseconds. Within this timescale, the robot must measure the state of the system, estimate the relevant dynamics, compute a corrective command, communicate it to the actuator, and physically produce the required motion.

This places significant requirements on sensing, computation, communication latency, and actuator bandwidth.

The robot considered in this thesis has a measured actuator bandwidth corresponding to a 2 Hz pole in servo mode, and up to 20 Hz in MIT mode depending on the position-loop gains.

This limitation will become relevant again when discussing the control architecture and the sim-to-real problem later in this thesis.

2.4 Reinforcement learning for locomotion

2.4.1 Actuator control architecture

Several actuator-level control architectures have been used in the literature. The main ones considered here are direct torque control, direct position control, and impedance control.

Direct torque control allows the policy to command the actuator torque directly, giving it the highest level of control authority.

Direct position control allows the policy to command a position target, which is then tracked by a low-level controller.

Impedance control allows the policy to command a desired mechanical behaviour through parameters such as position, stiffness, damping, and, depending on the implementation, torque-related terms.

The latter provides a compromise between control authority and the requirements imposed on the hardware in terms of latency and bandwidth.

Direct torque control provides the policy with the highest level of flexibility, but also imposes demanding communication and control-loop timing requirements.

In simulation, the policy can directly command joint torques at a high frequency [7].

On the physical robot, however, closing such a fast loop requires the policy, communication bus, actuator controller, and state estimator to operate with sufficiently low latency.

In the present hardware architecture, this approach was not practical through the CAN communication bus at the required frequency.

Direct position control substantially reduces these requirements by delegating the low-level actuator dynamics to the motor controller.

Position targets passed to the joint controller are used by several learned locomotion approaches [25, 21].

Two variants were investigated in this thesis: conventional smart-servo position control and the MIT control mode [10].

Both allow the RL policy to operate at a lower frequency while the actuator controller handles the faster inner control loop.

Impedance control provides an intermediate level of control authority [4].

Instead of directly specifying actuator torque, or being restricted to position targets, the policy can influence the desired mechanical behaviour through position, stiffness, damping, and/or torque-related parameters.

This avoids the need to close the torque loop at high speed over the communication bus while providing the policy with greater freedom than pure position control.

It also exposes more of the actuator dynamics to the policy and can reduce the sim-to-real gap compared with a smart-servo interface when the frequency and parameters of the inner control loop are not fully exposed.

2.4.2 Gait prior

The literature can be organised along a second dimension: the amount and location of prior structure imposed on the gait.

A strong prior can be introduced through a **reference trajectory**.

In this case, the policy is primarily trained to track an authored motion. This approach was, for example, explored on the Cassie robot [28].

This strongly restricts the reachable set of motions and can substantially simplify learning.

However, the quality of the resulting behaviour is bounded by the choice of reference trajectory: there is no guarantee that the selected trajectory is optimal for the particular robot morphology and dynamics.

A less restrictive prior can instead be introduced through a **trajectory generator** whose parameters are modulated by the policy [9].

This preserves the assumption that locomotion should have a structured, periodic form while allowing the learned controller to search for a gait adapted to the plant.

Trajectory generators can themselves provide different levels of freedom over the gait parameters.

At the stricter end of this generator-based formulation, the policy only modifies a small number of **high-level gait parameters**, such as phase, frequency, stride length, or amplitude [3].

This is similar to the reference-trajectory approach in the sense that the policy operates within a predefined family of trajectories. However, instead of learning the low-level joint commands required to reproduce a reference gait, the policy learns the high-level parameters that generate it.

This provides a relatively small search space but limits the set of gaits that can be represented.

A more moderate prior allows the policy to modify the **complete trajectory**. Rather than predicting every trajectory sample independently, the possible trajectories can be represented using a low-dimensional basis, such as Fourier series [23] or Bézier curves.

The policy then modifies the coefficients of this representation.

This retains useful structural assumptions such as periodicity and smoothness, while allowing the shape of the gait itself to become a learned variable.

It is also possible to use **neither a trajectory generator nor an explicit gait-shape prior**.

In this case, the policy learns the complete joint trajectory directly [6].

Periodic locomotion can still be encouraged through reward shaping, but periodicity then becomes an emergent property rather than an explicit constraint.

Such an approach provides a larger reachable set, but considerably increases the exploration and learning problem. There is also no guarantee that the learned solution will converge to a periodic gait.

Finally, the prior, when present, can be introduced at different levels: through the **reward** [24], through a **reference motion** [17], or through an authored **trajectory generator** [9, 3].

These choices determine how much of the gait is fixed and how much remains available to the policy at run time, at the expense of increasing or reducing the complexity of the learning problem.

2.4.3 Curriculum design

The third major design dimension is the curriculum used to train the policy.

To help the policy converge, the training environment can be modified to make the task easier at the beginning and gradually increase its difficulty as the policy improves.

The literature has explored a variety of curriculum-design choices, including the order in which different tasks are presented, the rate at which difficulty is increased, and the specific parameters that are modified to create the curriculum.

Terrain and command curricula. A common scheme starts every agent on flat ground with a narrow command range.

As its performance exceeds a predefined threshold, the agent is promoted to rougher terrain or a wider velocity range, and can be demoted otherwise [21].

Difficulty therefore tracks each agent’s competence rather than following a fixed schedule.

This approach is well suited to massively parallel training, where each agent can be promoted or demoted independently.

Automatic domain randomisation. Rather than fixing the domain-randomisation range before training, the range can be widened automatically once the policy exceeds a performance threshold at its current extremes [16].

The randomisation range itself becomes the curriculum parameter, expanding only as fast as the policy can tolerate.

This approach requires a reliable in-training performance signal.

A noisy signal can cause the range to grow faster than the policy can adapt, destabilising training.

Whether the curriculum targets terrain, command range, or randomisation range, an adaptive schedule removes the need to manually tune a progression rate.

In its place, the design requires a performance threshold and a metric with which to evaluate it.

This introduces a problem similar to that encountered in reward design: if the metric does not accurately represent the intended notion of competence, the agent may optimise the metric rather than the desired behaviour.

2.5 Platforms, and where DASH-01 sits

The published platforms fall into three families.

The first is the series-elastic research biped. Cassie, which held the 100 m record at the start of this project, is a purpose-built spring-mass machine with an institutional budget [5, 28].

The second family descends from the MIT Mini Cheetah [10]: a high-torque-density motor behind a small reduction, commanded in torque mode. This quasi-direct-drive pattern now dominates new legged platforms. The humanoids that took the sprint record from Cassie in 2026 carry the same family of joints [2].

The third family is the hobby-class humanoid built from position-controlled smart servos. The servo interface hides the actuator dynamics from the controller, and these machines remain

confined to quasi-static walking.

DASH-01 stands in the second family by principle and by cost structure: quasi-direct-drive actuators, light legs and a commodity computer. It differs in the command interface, since its drives are commanded through their internal position loop rather than in torque. Section 3.4.7 measures the cost of that difference.

2.6 The gap

The learned-locomotion results above share a hardware assumption: the platform offers high-bandwidth torque or impedance control at the joint [7, 10, 28].

How much of this methodology survives on a low-cost plant, commanded through a position-servo interface of limited bandwidth, is not documented.

This thesis explores that gap on DASH-01. Section 3 therefore starts by finishing and measuring the plant, so that the training and its results are judged against the machine as built rather than as designed.

3 The robot and its digital twin

This chapter turns the inherited mechanical design into a measured plant. The non-mechanical subsystems are selected first: actuators, battery, compute, sensing and the communication bus. A digital twin is then derived from the CAD data and validated against the assembled robot. The parameters the CAD cannot provide, such as the actuator bandwidth, are measured on the real robot to limit the sim2real risk. The chapter closes with the running speed ceiling the measured plant admits, which Section 4 trains against.

3.1 DASH-01

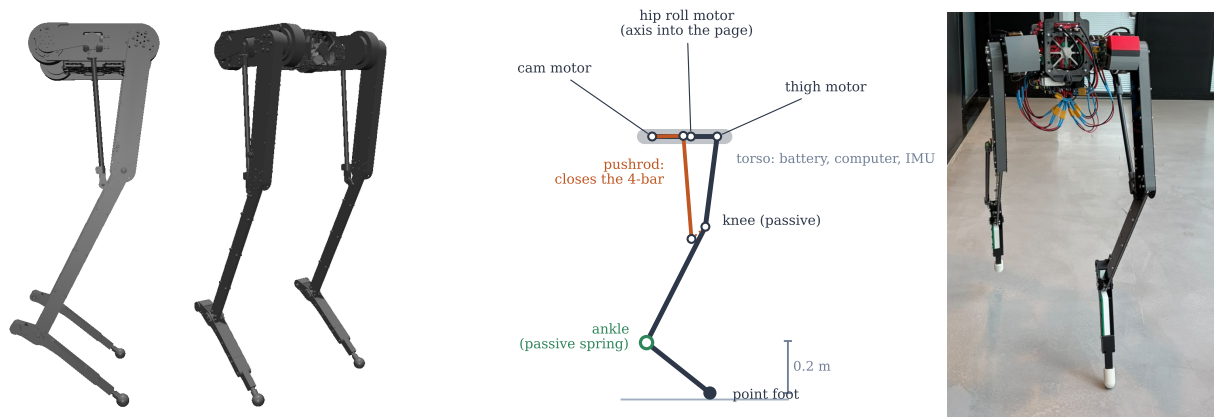


Figure 3.1: DASH-01, Simulated model and real-life picture.

	Hip roll	Cam, thigh		
Part	AK60-39	AKE90-8	Segment	Mass Count
Reduction	39:1	8:1	Base	5764 g 1
Peak torque, datasheet	72 N·m	170 N·m	Hip, with two motors	3271 g 2
Peak torque, measured	61.2 N·m	144.5 N·m	Cam	66 g 2
Continuous torque	24 N·m	55 N·m	Thigh	483 g 2
No-load speed	10.3 rad/s	22.0 rad/s	Push rod	71 g 2
K_t (N·m/A)	0.12	0.272	Ankle spring assembly	249 g 2
R (Ω)	0.60	0.164	Shin	324 g 2
K_m (N·m/ $\sqrt{W}$)	0.15	0.674	Foot	222 g 2
Reflected armature (kg·m ²)	0.046	0.0216	Total	15 136 g
Mass (kg)	0.75	1.40		
Unit price (EUR)	393	424		

Table 3.1: The six CubeMars quasi-direct-drive actuators (left) and the weighed segment masses (right). Measured peak torque is the datasheet figure scaled by the effective joint K_t of Section 3.4.2. K_t and R are line-to-line, and the motor figure of merit is $K_m = K_t/\sqrt{R}$.

Each leg carries three motors. Hip roll sits at the base, and a cam and a thigh axis drive the sagittal plane. The knee is passive and follows through the 4-bar. The ankle is passive and follows through a spring (not rendered). The spring was later measured to be roughly an order of magnitude too soft: at half body weight it deflects 26.6 deg, which would drive the heel 11.9 mm through the floor. The ankle was therefore locked in a neutral position with a carbon tube, and the simulation masses were updated to reflect the change.

3.2 Selecting hardware

3.2.1 Actuators choice

A database to select the actuator was assembled. Major robotic actuator manufacturers website was scraped for datasheet and a large language model was used to parse each datasheet. The resulting database holds 140 entries across 11 suppliers, listed in Table 3.2.

Manufacturer	Actuator count
ZeroErr	32
MyActuator	29
CubeMars	22
SteadyWin	21
Maxon	12
MAB Robotics	8
RobStride	7
HEBI Robotics	6
Soceboz	1
RSL-ETH	1
mjbots	1
Total	140

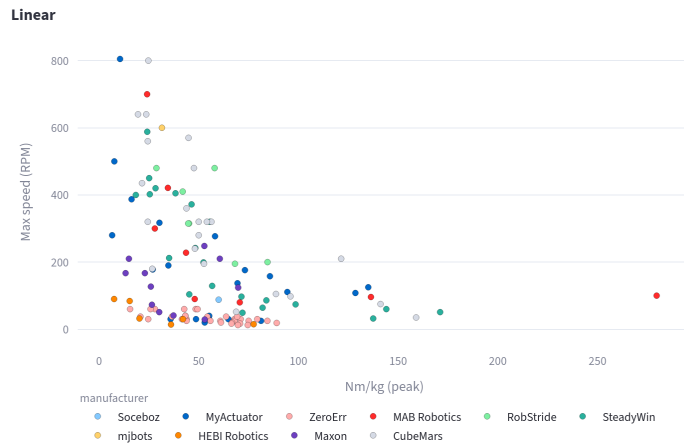


Table 3.2: Actuator manufacturers represented in the database

Figure 3.2: Excerpt of the dashboard, showing the actuator database used to select the drive.

The database was compiled into an interactive dashboard, *motormap* [14], built from the `actuatorReview` code [13]. The choice of the actuator itself is detailed in the master’s thesis of M. Aebi [1]. The final motor choice was the CubeMars AK60-39 for the hip roll and the AKE90-8 for the cam and thigh. The AKE90-8 is rated for up to 72 A peak, while the AK60-39 is rated for 17 A peak.

3.2.2 Battery choice

A database of roughly 430 commercial cells was built on the same principle as the actuator database. Since 48 V is the highest pack voltage the selected drives accept, the database was filtered to cells usable in a 13S configuration. The motor selection sets the current requirement. Assuming half of the six motors at peak torque simultaneously (two AKE90-8 and one AK60-39), the pack must deliver about 160 A. From these candidates, a 13S2P pack of EVE INR21700-50PL cells was selected, as the best compromise between energy density, power density and cost.

The pack can be monitored over Bluetooth and its limits configured there with the DALY BMS app. A CAN interface is also available but was not used, due to lack of time.

The manufacturer states the internal resistance below 10 mΩ per cell, which gives about 65 mΩ for the 13S2P pack. At the rated 360 A peak, the voltage drop would reach 23 V. At the 160 A working peak of the motor budget, the drop is about 10 V, which the BMS low-voltage protection tolerates on a full charge. Due to the lack of voltage feedback on the motor drivers, this figure has not been verified on the real robot.

Parameter	Specification
Cell	EVE INR21700-50PL, 5.0 Ah
Configuration	13S2P
Nominal voltage, capacity	48.1 V, 10 Ah (468 Wh)
Continuous rating	100 A, cell-limited
Peak current	360 A for 2 s
Mass (BMS, casing)	3.5 kg
Power density (peak)	4900 W/kg
Power density (cont.)	1370 W/kg
Energy density	133.7 Wh/kg

Table 3.3: Specification of the selected 13S2P battery pack.

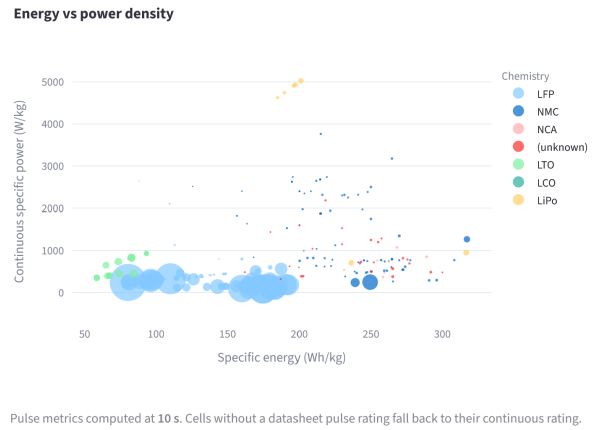


Figure 3.3: Excerpt of the dashboard, showing the cell database used to select the pack.

3.2.3 Compute, CAN bus and sensing

The following Table 3.4 summarises the choice of the compute, sensing and network bus.

Compute	
Board	Raspberry Pi 3B, stock non-real-time Linux
Control loop	200 Hz
Sensing	
IMU	ICM-20948, 10-axis, on a Waveshare Sense HAT (B)
IMU interface	I2C, ± 4 g, ± 1000 dps, 200 Hz sampling
Analog input	4-channel ADC on the same HAT
Network bus	
Controller	mcp251xfd over SPI, dual-channel CAN HAT
Topology	two independent 1 Mbit buses, one per leg
Measured load	26 % of each of the two buses (polled at 200 Hz)

Table 3.4: The robot’s non-mechanical architecture: compute, sensing and the CAN network bus.

The board is a **Raspberry Pi 3 Model B**, running stock non-realtime Linux. It was chosen over the laboratory’s usual BeagleBone Black for its faster SSH sessions and its four cores, which let the control loop and the data processing run side by side. An **mcp251xfd** controller over SPI provides CAN as two independent 1 Mbit buses, one per leg. The Pi also hosts a WiFi access point and the web interface of Section 3.5, through which most measurements in Section 3.4 were taken.

Section 3.4.4 presents the measured timing of the control loop and the CAN bus.

Aside from the motor encoders and current feedback, the only proprioceptive sensor is the IMU located on the base of the robot. Contact sensors were considered and never fitted due to lack of time. The IMU unit is an **ICM-20948** on a Waveshare Sense HAT (B), read over I2C.

3.3 The digital twin

A simulation ready model was derived from the CAD data. The model is exported in two formats, URDF and MJCF. The detail procedure to export the model is described in the

appendix Section 8.1. This took a lot of time and effort due to lack of experience and the poor documentation of the plugin used to export the model. The final recipes consisted into exporting each segment of the robot as a STEP file with the origin at the joint (except for the base which was in its center). Care was taken to ensure the references axis were aligned in a consistent manner across the segments. The robot model was reassembled in autodesk Fusion 360 and exported to URDF using the ACDC4Robot plugin[18]. The kinematic loop where broken in the URDF and closed in the MJCF using an equality constraint between two sites, one on the pushrod and one on the shin. This was necessary because URDF is a tree format and cannot express kinematic loops.

3.4 Digital twin validation and plant identification

The digital twin is validated against the real robot for the kinematic and dynamic properties. The model is also refined with measured parameters that cant be accurately estimated from the CAD model, to reduce the sim2real gap.

3.4.1 Joint Limits validation

The cam and thigh joints close a passive 4-bar loop through the pushrod and knee. Their safe ranges do not compose into two independent intervals: a per-joint minimum and maximum would allow poses the mechanism cannot assemble. The safe envelope is instead derived directly in actuator space. Each leg's three drives are held current-free and the leg is backdriven by hand through its full travel, logging the raw actuator angles [15].

Figure 3.4 shows the result for both legs: a plain interval for hip roll, and a two-dimensional band for the coupled (cam, thigh) pair. The demonstrated hip roll range spans 42.8 deg to 126.1 deg on the left leg and 66.5 deg to 147.7 deg on the right; both are eroded by 3 deg for a safety margin and the eroded interval is stored as the hip roll limit. For the knee pair, the recorded (cam, thigh) samples are rasterised onto a 1 deg grid, dilated to close small sampling gaps, then eroded by the same 3 deg margin. The stored safe region covers 11.0 % of its own bounding box on the left leg and 9.4 % on the right, confirming the band is far from rectangular. The onboard safety layer checks every commanded pose against this grid before it reaches the drives (Section 3.5.2).

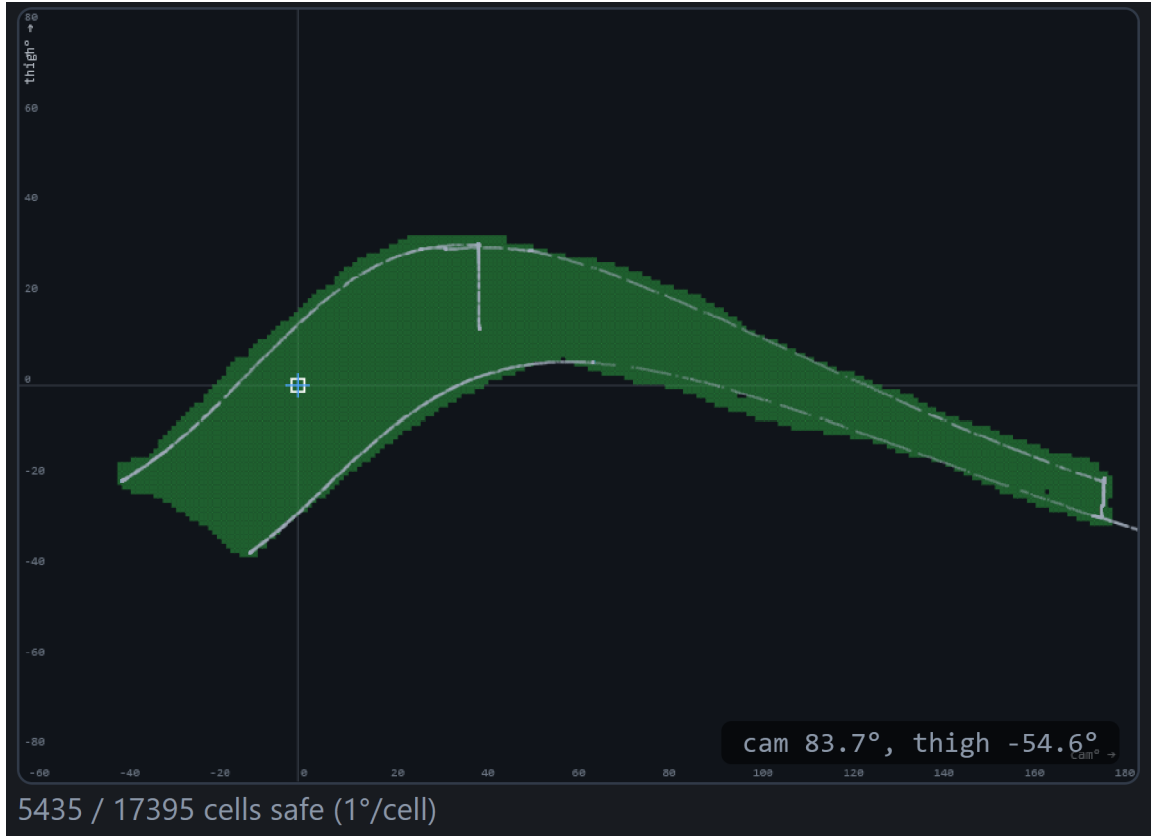


Figure 3.4: Joint-limit validation in actuator space: the coupled (cam, thigh) 4-bar band, backdriven by hand for each leg. The green region is the safe envelope enforced by the onboard safety layer.

3.4.2 Inertia matrix validation

Each actuated joint’s torque constant K_t , effective inertia and friction (viscous and Coulomb) are identified from two excitation profiles logged through the web interface [15]. A quasi-static sweep at 0.03 Hz holds the joint near equilibrium, so gravity dominates the measured torque. A single per-joint least-squares fit on the pooled data recovers K_t , the effective inertia and the friction terms together. The fit uses the model’s gravity torque, computed through the closed 4-bar loop (Section 3.3).

A raw torque residual is hard to judge on its own, so it is converted into a mass-equivalent error. It is an upper bound, since a part of the residual is friction rather than gravity torque. Figure 3.5 reports the per-joint values. The relative error is large because the segment masses are small, so their gravity torque sits close to the noise floor. The check still serves its purpose: it confirms that the inertia matrices are aligned in the correct frame and of the correct magnitude, and that the gravity trend is right. Since the CAD values are not random, a correct frame and magnitude imply the export transformation is correct, and the CAD inertia data can be trusted. The bounded mass error is reused as a domain-randomisation range in the RL training, to make the policy robust to this uncertainty.

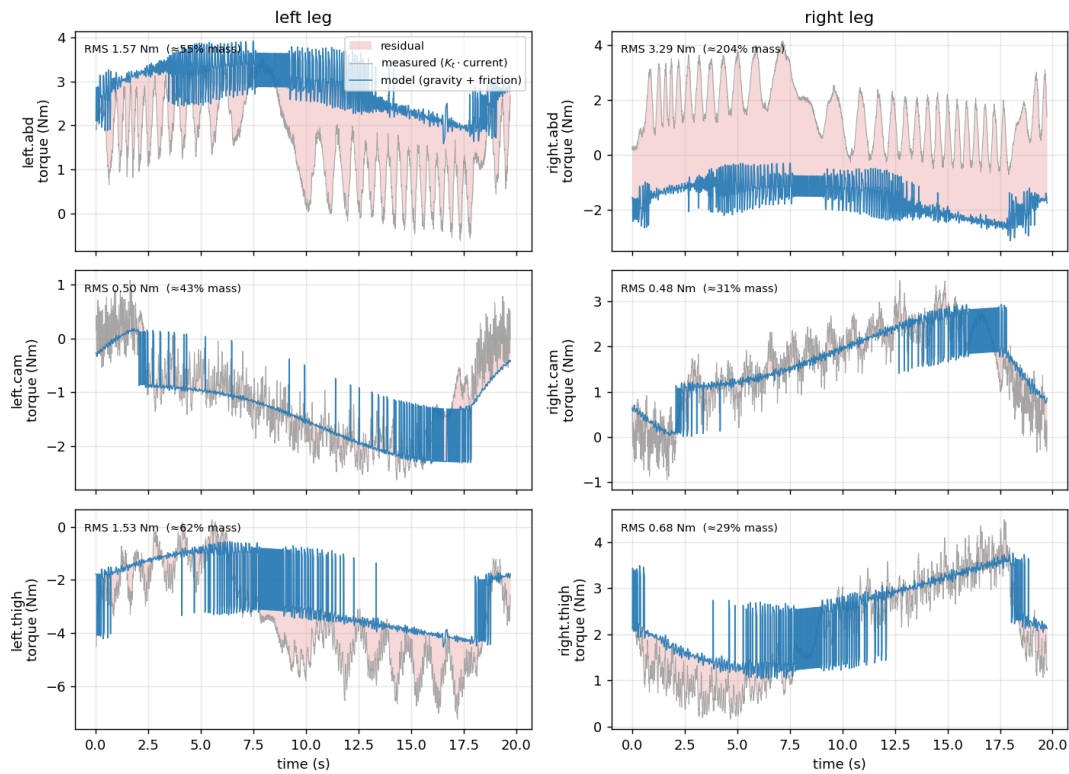


Figure 3.5: Gravity-model validation on the slow (0.03 Hz) quasi-static sweep: measured torque (K_t times current) against the model's predicted torque (gravity plus friction), residual shaded, per joint. The mass-equivalent error is the RMS residual divided by the RMS gravity torque. The fast ripple is likely the P-only position loop's stick-slip response due to friction.

3.4.3 IMU characterisation

The base IMU was sampled at rest for 300 s, configured at $\pm 4g$ and ± 1000 dps, with a 50 Hz internal low-pass and 200 Hz sampling. The measurement gives 1.8 mg and 0.08 dps RMS noise. The noise density is $218 \mu g/\sqrt{Hz}$ and $0.0096 \text{ dps}/\sqrt{Hz}$. The gyro turn-on bias is repeatable at $[1.03, 0.11, -0.53]$ dps, and the in-run bias stability is 0.010–0.012 dps.

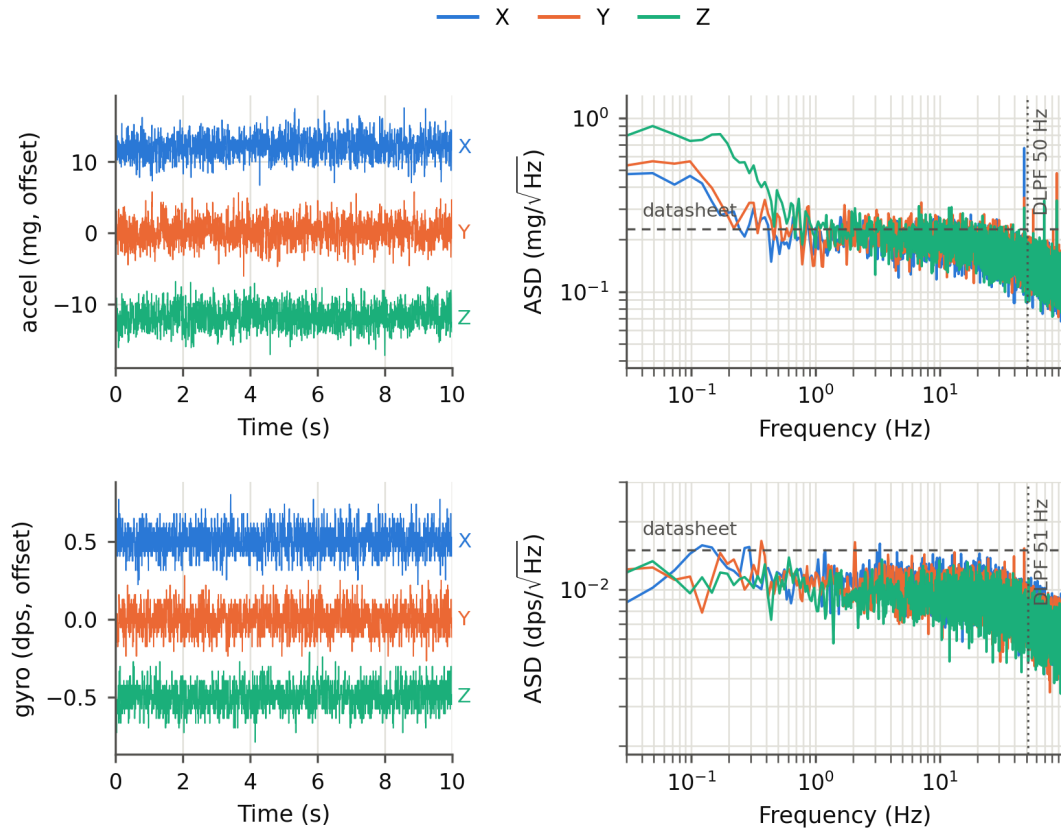


Figure 3.6: Noise characterisation of the base IMU (ICM-20948), sampled at 200 Hz for 300 s at rest.

3.4.4 CAN bus latency and load characterisation

The whole control loop was benchmarked on the robot with the web interface running. These results were reused in the RL curriculum to make the policy robust to the timing jitter.

Rate	Mean period	Jitter (sd)	p99	Max	Budget used
50 Hz	20.148 ms	2.151 ms	22.41 ms	49.29 ms	23 %
100 Hz	10.000 ms	0.251 ms	10.40 ms	12.28 ms	37 %
200 Hz	5.000 ms	0.141 ms	5.21 ms	7.46 ms	50 %
400 Hz	2.504 ms	0.193 ms	3.24 ms	5.13 ms	73 %

Table 3.5: Measured closed-loop timing on the robot’s Raspberry Pi 3B, with the web interface competing for CPU. One tick drains both CAN buses, runs the inference of the policy and writes all the motors commands.

At 50 Hz the mean period overshoots its 20 ms target and the jitter is an order of magnitude above the faster rates. The likely cause is the non-real-time scheduler, which wakes the loop late after the longer sleeps.

3.4.5 Actuator thermal characterisation

Not measured due to lack of time. The thermal lumped model parameters were estimated roughly from the mass of the actuator but not used in the simulation.

3.4.6 Battery characterisation

Not done due to lack of time, estimated from the data sheet internal resistance of the cell. This is further complicated by the lack of voltage feedback on the motor drivers, which prevents measuring the voltage drop under load. In practice, this is not expected to be a problem: the drives are not used in torque control, so they can be expected to regulate the voltage drop without taking the RL policy out of its distribution.

3.4.7 Actuator bandwidth characterisation

The AKE90-8's closed-loop bandwidth was measured in both Servo and MIT mode.

Figure 3.7 shows the results. Servo mode reaches -3 dB at 1.95 Hz. MIT mode reaches -3 dB between 6.0 Hz and 20 Hz depending on k_p and k_d .

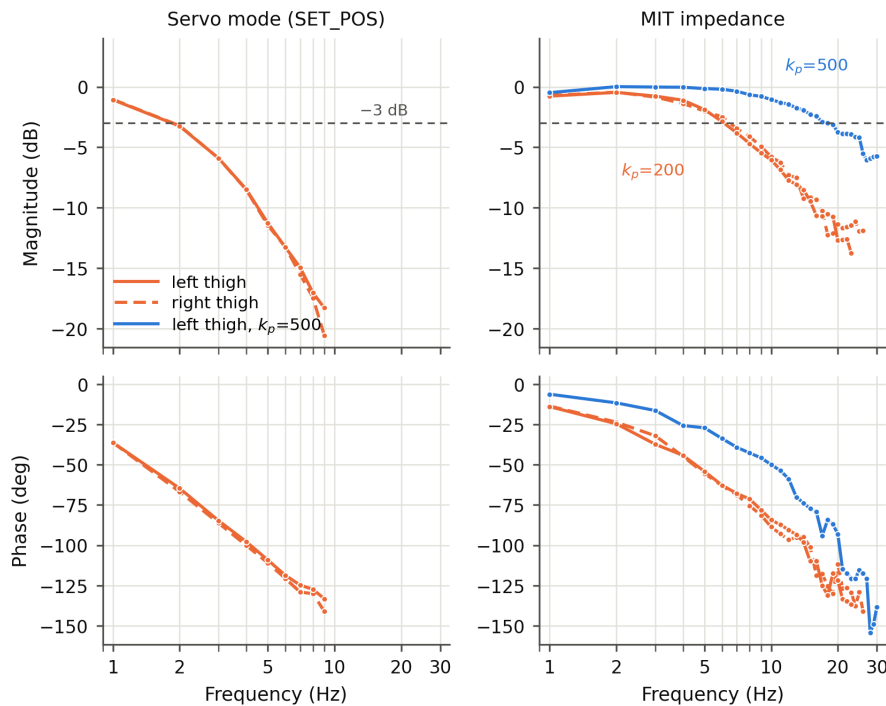


Figure 3.7: Measured closed-loop tracking a sine chirp and its response as the frequency increases in term of tracking error of the AKE90-8 drive: servo mode's position loop (free rotor) against MIT mode's position loop (free rotor).

3.5 Deployment stack

The policies of Section 4 have to run somewhere. To streamline the process and simplify the measurement of the plant parameters, a web interface was built on the robot. A systemd entry starts the web interface at boot. The Raspberry Pi acts as a WiFi access point (SSID "runningmachine", password "runningmachine"). Once connected to the network, the web interface is reachable at <http://192.168.4.1:8080/>.

3.5.1 Calibration sequence

The actuator don't hold their position when powered off, so a calibration sequence is needed to zero the joints. The user need to hold the robot in a known configuration and run the calibration

sequence which register the current joint position as zero. The user then need to move each joint in a specific direction and record that the angle in increase (otherwise, the joint direction need to be inverted by clicking a button in the web interface). Once the robot is zeroed, the workspace can be registered by backdriving the robot and recording the joint position at the limit of the workspace. The web UI then clean the measurement to show the boudany and the user can draw / fill the safe workspace in the web interface. This is used to limit the joint position in the safety layer.

the IMU is also calibrated in the web interface by holding the robot still and recording the bias of the accelerometer and gyroscope. To ensure the axis are aligned, the user need to move the robot around an given axis of the base and check that the correct axis is increasing.

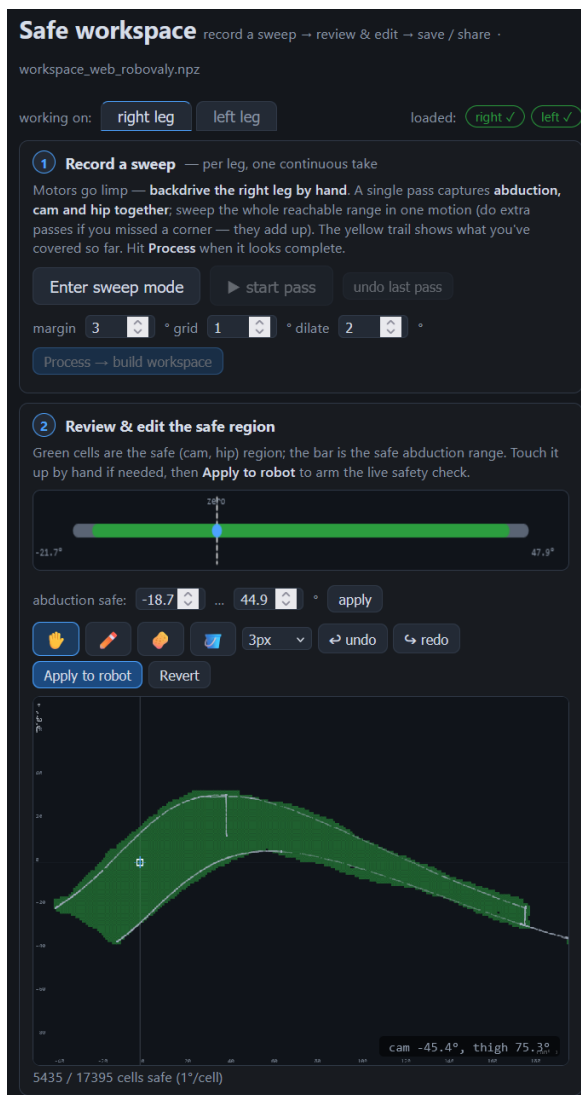


Figure 3.8: Workspace widget of the web interface, showing the backdriven joint envelope and the user-drawn safe workspace boundary.

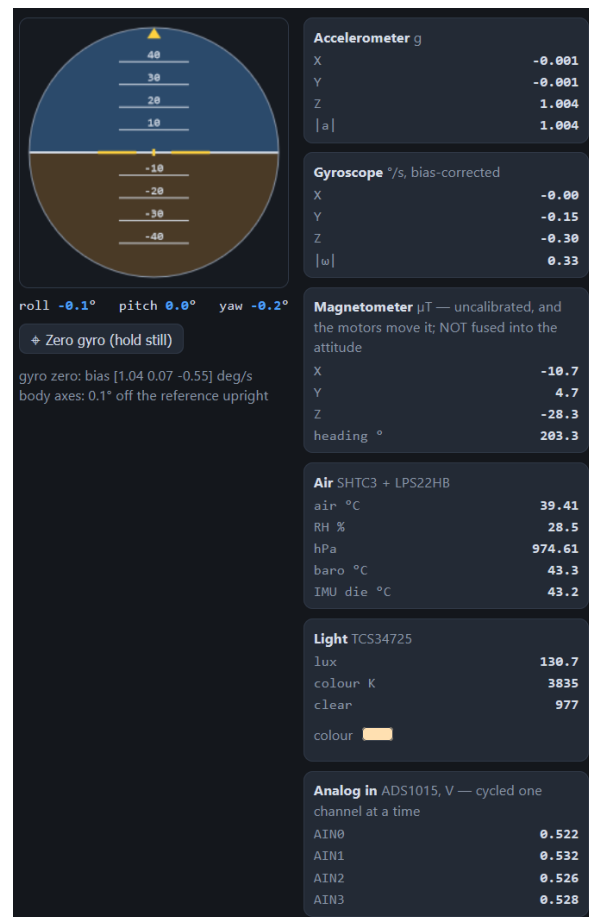


Figure 3.9: IMU widget of the web interface, used to record accelerometer and gyroscope bias and check axis alignment.

3.5.2 The safety layer

A safety layer was implemented to prevent the robot from exiting the safe operating envelope. It limits the joint position, the joint speed, the joint torque. If a tracking error greater than a

threshold is detected, the robot is stopped. The user can also at any time stop the robot by clicking the E-STOP button in the web interface. This safety layer was thoroughly tested in simulation and on the real robot by trying to make the robot exit the safe operating envelope and was proven useful in a few occasion to prevent user error. The safety layer also implement a black box recorder that record the joint position, speed, torque and the IMU data at 200 Hz if a new failure mode is triggerd, it will be possible to investigate it.

3.5.3 Running widget

From the web interface it is possible to both record a gain by backdrivign the foot of the robot and then to play it back at a wanted frequency. This is also where a policy can be selected and run on the robot.

3.5.4 Measurement widget

Three widgets carry the plant identification campaign of Section 3.4; Figure 3.10 shows them. The *thermal identification* widget deposits a known amount of electrical heat in one motor and records the case temperature, to fit a lumped thermal model. The *dynamic identification* widget excites one leg with workspace-checked position chirps and logs position, velocity and current at 200 Hz. The recorded runs feed the fit of Section 3.4.2. The *limbs and inertia* widget stores the weighed segment masses and compares the CAD inertia tensors against the identified ones. The comparison uses the principal moments, which are frame-invariant, so wrong values are told apart from a wrong export frame.

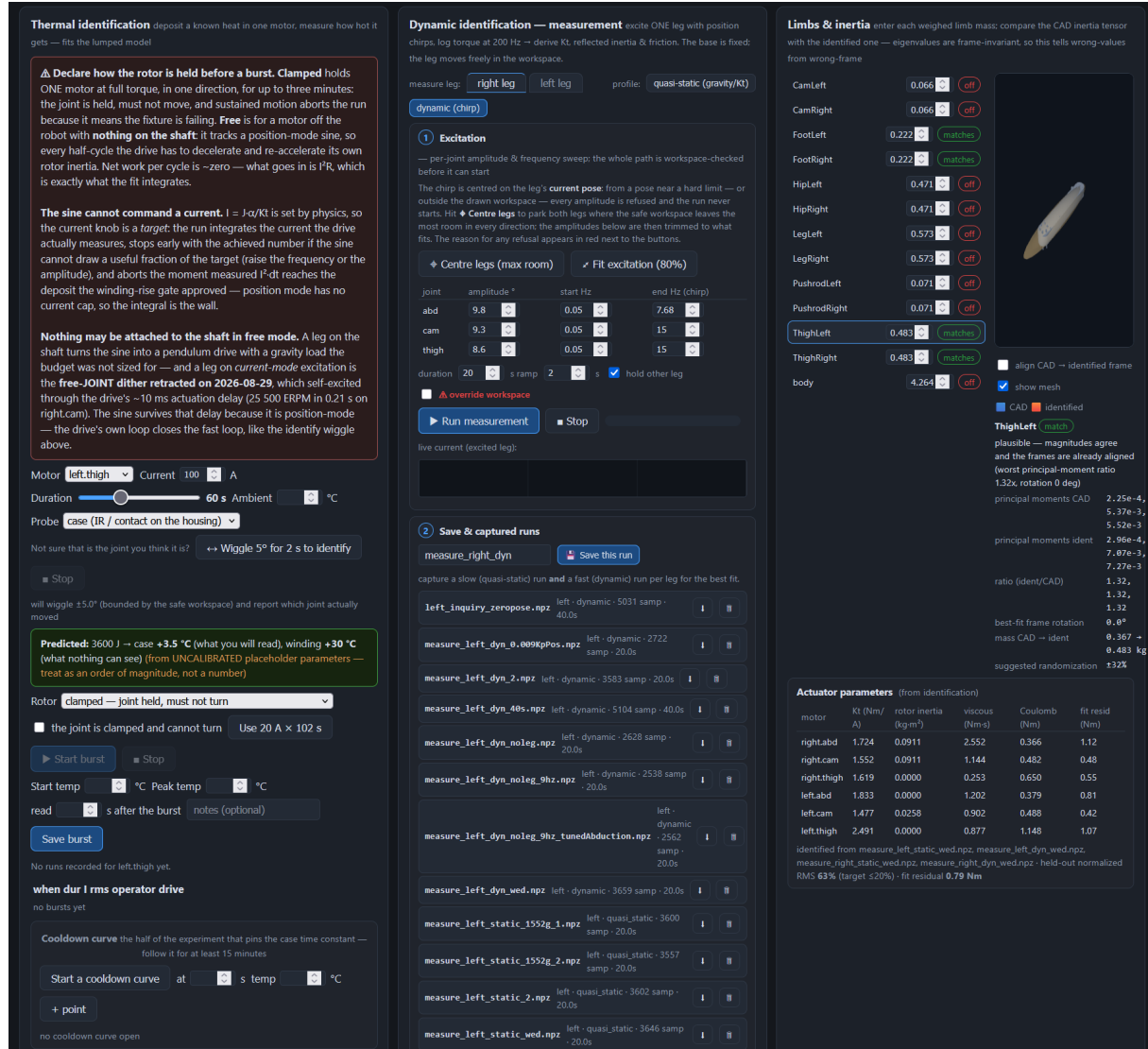


Figure 3.10: The three widgets used to validate the robot parameters and limit the sim2real gap.

3.6 Running speed ceiling estimation

The physical speed ceiling was established independently of reinforcement learning by applying progressively stricter physical constraints to the same robot model. First, the body was fixed and ground interaction removed with the leg free to move. This was used to determine the maximum feasible fore-aft foot sweep over the empirically verified workspace. The resulting speed was evaluated first from joint-speed limits alone, then with the measured torque-speed envelope and leg inertia, and finally with the measured actuator command latency. This progressively reduced the theoretical ceiling from 14.5 m/s to 7.4 m/s and ultimately 6.55 m/s, showing that actuator torque and leg inertia are the dominant limitations. The result is an optimistic upper bound because the leg carries no body load and balance and contact constraints are ignored.

A second analysis extends the bound to gaits involving a flight phase, where the rail argument no longer applies because ballistic flight can increase stride length. The maximum speed was instead constrained using an impulse balance: the available vertical foot force, obtained from the leg's force capability, was combined with the minimum time required to execute a stroke. As speed increases, stance duration decreases while the leg's minimum cycling period remains finite,

eventually leaving insufficient stance impulse to support the robot’s weight. This gives a slightly lower upper bound of approximately 6.2 m/s. Both analyses agree on a ceiling of about 6 m/s, and this rounded figure is the bound quoted in the rest of the document.

Model	Bound	Cost	Binding mechanism
Kinematic (no-load speed only)	14.5 m/s	–	joint speed through path Jacobian
+ torque–speed envelope, inertia	7.4 m/s	–49 %	stroke reversals, torque at speed
+ 7 ms MIT-mode latency	6.55 m/s	–12 %	delay caps usable stiffness
Flight-phase closure (any gait)	6.2 m/s	–	vertical-impulse starvation

Table 3.6: The speed-ceiling with refined modeling incorporating more realistic physical constraints.

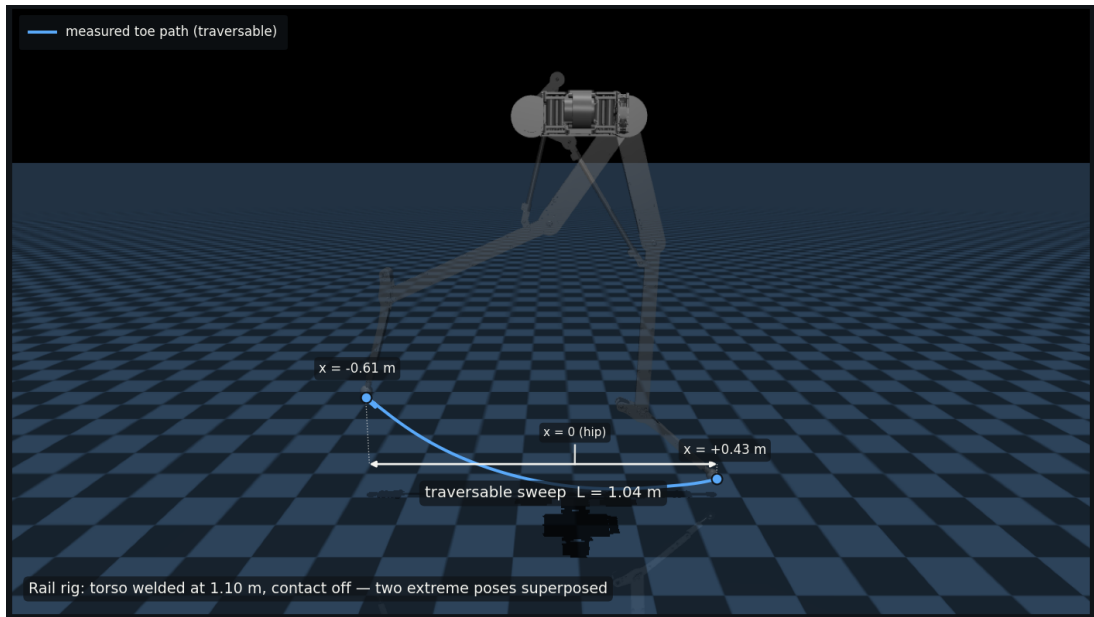


Figure 3.11: Workspace of the leg used to infer an optimistic upper bound on the running speed.

3.6.1 Static toe-force map

The speed bound of Section 3.6 needs the force the leg can put on the ground. The leg’s static force capability over the measured workspace is therefore mapped.

The domain is the backdriven workspace of Section 3.4.1. The toe Jacobian comes from the model’s closed-loop forward kinematics (Section 3.3), differentiated over a 1 deg grid. The joint torques are capped at the measured 144.5 N·m of Table 3.1. The maximum pure-vertical force F_z and pure-horizontal force F_x at each pose then follow from $\tau = J^\top F$. Vertical values are clipped at 3.5 body weights as near full extension the leg acts as a strut, and the Jacobian alone would admit an arbitrary vertical force there. Poses past the 4-bar’s fold boundary are cut, since force there is not usable.

Figure 3.12 shows the result in body weights (15.14 kg). The capability is strongly anisotropic. At the standing pose, near 96 % of reach, the force ellipse is about 19:1: the leg pushes hard straight down and weakly fore-aft, which is suboptimal for propulsion. The vertical map feeds the impulse bound of Section 3.6, and the anisotropy makes the linkage proportions a design lever that is presented in Section 6.

Static end-effector force capability over the recorded safe workspace (values capped at 3.5 BW; colour scaled to each map's 10th--90th percentile)

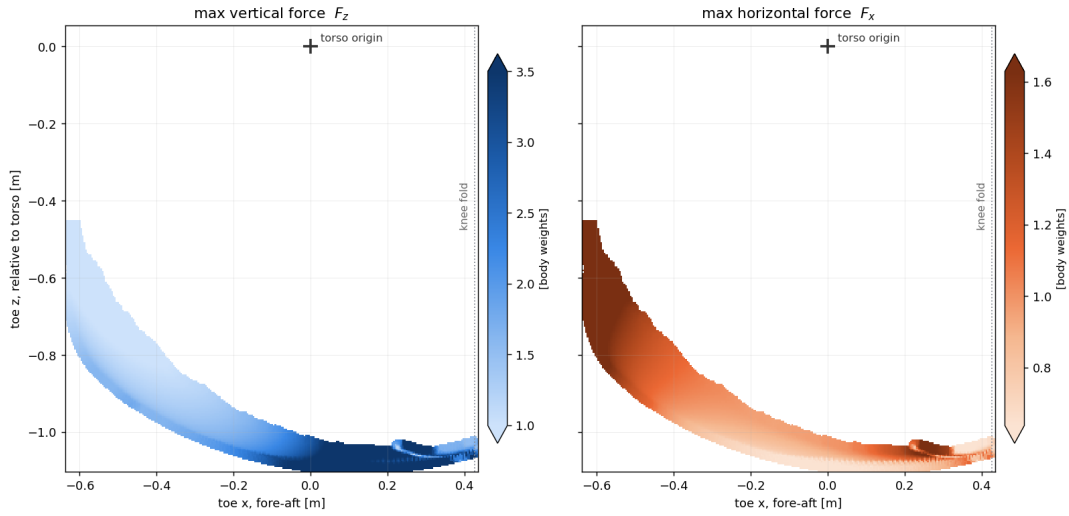


Figure 3.12: Static toe-force capability over the measured workspace: maximum vertical force F_z (left) and horizontal force F_x (right), in body weights, clipped at 3.5 BW. The vertical map feeds the flight-phase impulse bound.

3.6.2 Sensitivity of the bound to the design parameters

The latency-included rail model also gives the sensitivity of the bound to the design. Four parameters were each scaled by $\pm 30\%$ with everything else nominal: leg mass, rotor inertia, deliverable peak torque and bus voltage (through the no-load speed). Peak torque is the strongest lever: 30% more raises the bound by about 1 m/s. Leg mass is second: 30% less raises the bound by about 0.7 m/s. Rotor inertia is nearly flat, and 30% more bus voltage gains nothing, because the bound does not sit on the no-load-speed ceiling at this operating point. Leg length was not simulated due to lack of time as it imply to update the URDF but it is assumed that longer leg would help increase the stride length. The levers for a faster machine are therefore deliverable torque and leg mass. Leg length is also likely a significant level but its impact and direction is unknown. Section 6 ranks the mechanical changes that follow from them, and Section 4 trains its policies against the plant measured in this chapter.

4 Reinforcement learning: attempts and architecture

Section 3 supplied a measured plant model. This chapter develops the controller for it. Multiple architectures were attempted, starting from raw torque commands and progressively adding structure to the action space. The main changes were motivated either by training failures or by limitations identified on the real robot.

The learning algorithm itself was kept fixed. The changes therefore concern the formulation of the task, the action representation, the curriculum and the amount of structure given to the policy. The final architecture is a Fourier gait generator with impedance parameters and reflexes.

4.1 Formulation

The task is a single dash: run as fast as possible.

Several formulations were attempted during the project. The first formulation only required the robot to run forward, without requiring it to stop at the end of the track. Later formulations attempted to add velocity command for a joystick input. These formulations were more general, but also substantially harder to train. Since the objective of this thesis is maximum running speed rather than general locomotion, the final experiments used only the simpler dash formulation.

The learning algorithm was held fixed at PPO [22] from Stable-Baselines3, with an actor-critic MLP running on CPU-parallel MuJoCo environments. Episodes are capped at 60s. The control rate is 200 Hz, which Section 3.4.4 establishes as both the sensing ceiling and the maximum deployable rate. Later experiments showed it to be borderline for the final controller and the frequency need to be brought back down to 100Hz as the final policy inference via the web interface with the safety layer proved a bit too slow for the 200Hz control loop on the Raspberry Pi 3B. The final policy therefore needs to be retrained to run at 100Hz. The computational budget was large but finite and within what can be seen in the literature. A milestone corresponds to approximately 600 M environment steps and a multi-day cluster job. The thesis as a whole used approximately 15 G environment steps, with an estimated 75 Kg of CO₂eq emissions from the cluster usage. To help put that into perspective, the author exhaled 150 kg of CO₂eq during the period of this thesis from respiration or a simple way airplane trip from Geneva to Barcelona.

4.2 The toy problem

Committing a 12-DOF robot to an unproven learning and deployment pipeline is risky. The complete chain was therefore exercised once at minimum stakes, on a single-motor inverted pendulum. The purpose of this experiment was not to study locomotion. It was to verify the complete path from simulation to the physical controller. The chain consists of simulation, training, export, deployment on embedded Linux, and a closed loop over CAN. The observation space was deliberately restricted to the telemetry available from the real drive. It contains the motor position in turns, velocity in turns per second and torque. The observation normalisation statistics used during training are exported as part of the ONNX graph rather than being reconstructed separately on the embedded system. The resulting policy runs on a beaglebone black using `onnxruntime`. The control loop closes over CAN at 50 Hz. This experiment therefore tested the software and it's author to help prepare for the full robot before committing the computational and experimental budget.

4.3 Observation space

The policy can see only measurements that can be produced by the hardware. The observation contains the joint position, joint velocity and joint torque, the IMU gravity direction and angular velocity, the commanded motion and the previous N actions and state.

The observations are expressed in physical units and passed through fixed normalisers. A history stack covering approximately 200 ms is used instead of giving the policy access to a simulated base-velocity measurement that would not be available on the physical robot. Sensor noise, bias drift and CAN staleness are injected according to the measurements from Section 3. This makes the observation presented during training closer to the signal available during deployment. The actor and critic use asymmetric observations. The actor receives only the deployable observation described above. The critic additionally receives a privileged simulation-only tail containing quantities such as the true base velocity and contact information. These privileged quantities are therefore used to improve value estimation but are not available to the deployed policy.

4.4 Reward design

The reward was built around forward progress rather than an alive bonus. A dense forward-progress term rewards the robot for moving towards the end of the track, while a per-step time cost makes standing still increasingly expensive. This distinction became important during training because an alive bonus rewards the robot for remaining upright even when it has stopped, which the policy learned to game when a joystick driven attempt was made. For a sprint task this is undesirable: a controller that stands indefinitely can obtain a positive reward without solving the actual task. The time cost instead makes a stationary solution progressively less attractive.

Termination is also part of the reward structure. Body contact and leaving the valid workspace terminate the episode. Consequently, the fall penalty is mainly the future reward that is lost when an episode terminates rather than a single large numerical penalty.

Reward design was repeatedly modified in response to exploitation by the policy. The robot learned to belly-flop, to move very slowly, to vibrate in place and to skate. The fall penalty was changed when the robot learned to exploit belly-flopping, the alive-related terms were changed when it learned to dawdle, and foot air-time and action-rate terms were introduced to suppress vibration and other non-locomotive solutions.

This was preferable to repeatedly changing the relative weight of all reward terms. Each additional term was intended to prevent a specific mechanism that had been observed on the robot or in simulation. Action-rate penalties were introduced to limit rapid changes in the command sent to the actuator. Gait-specification-rate penalties limit rapid changes in the parameters of the gait generator. Torque and energy terms discourage solutions that use unnecessarily large actuator effort. These terms therefore do not only regularise the policy numerically; they also constrain behaviours that would be undesirable on the physical robot.

4.5 Evaluation protocol

Training-time episode length is not used directly as the final performance metric. During exploration, the policy contains a non-zero action distribution, and a long episode can therefore result partly from the injected exploration noise. The results are obtained from greedy evaluations, in which the exploration noise is disabled and the policy deterministically executes its preferred action at each timestep. This measures the behaviour that the learned policy will produce once deployed, rather than its behaviour under exploration noise.

The experiments also showed that performance is not well described by a single mean over a small number of seeds. Runs were frequently bimodal: a seed either discovered a useful gait or remained in a non-locomotive behaviour for a large part of the available budget. Comparisons therefore use paired seeds and survivor counts where possible rather than relying on small unpaired averages.

4.6 The action-space ladder

Several action spaces were attempted, from the least structured representation to the final gait-generator representation. The progression is not presented as a hierarchy in which each later representation is universally better. Each one was tried to solve a problem and introduced a different limitation.

4.6.1 Direct joint torque

ADD ARCHITECTURE SCREENSHOT

The first framework commanded joint torque directly on a simplified biped, because the robot was still being designed. It was abandoned for two different reasons: the training did not converge, and the assumed hardware interface was not considered deployable at the required rate.

As written earlier, the training did not converge after many attempt at shaping the reward function. The fall penalty rose because the robot learned to belly-flop. The alive bonus changed because it learned to dawdle. The feet-air-time thresholds were retuned because it learned to vibrate in place.

The underlying difficulty is that a raw torque action space requires the policy to synthesise two separate behaviours simultaneously ; It has to generate the gravity compensation required to keep the robot upright, while also discovering the periodic structure required for locomotion. Between these behaviours, the reward landscape contains many ways of falling over that can nevertheless produce a temporarily favourable reward.

The second problem was the assumed deployment path ; Direct torque control bypasses the drive's internal position or impedance loop and requires current or torque commands to be streamed over CAN. A simple PID loop was tested over the CAN bus as a practical check. As expected, the leg could track slow points, but became increasingly jittery when the desired trajectory was played faster.

4.6.2 Joint position targets

ADD ARCHITECTURE SCREENSHOT

The policy was now expected to output six position targets for its action vector, one for each actuated joint. In simulation these commands drive MuJoCo position actuators with gains and force ranges derived from the actuator specifications. On the physical robot the action maps directly to the position targets accepted by the drives. Two hardware implementations were considered. A smart-servo type interface provides its own internal position loop, while the MIT interface exposes position, stiffness and damping parameters. Measurements in Section 3.4 showed a large bandwidth difference between these interfaces. And the PID loop that are cascaded in the motor driver are not known which make this approach a liability when it come to sim2real gap. The MIT mode consequently became the relevant interface for the final experiments. Furthermore, in one experiment the robot learned to skate while the reward curve continued to improve. This taught the author that that episode reward alone was insufficient to infer the success of a policy for locomotion. More reward shapping was attempted to penalize this behaviour with limited success.

4.6.3 Impedance with static gains

ADD ARCHITECTURE SCREENSHOT

After a significant ammount of time spent to understand how to send command to the CubeMars drives in impedance mode to avoid the actuator bandwith issue in servo mode, the action space was extended to include stiffness and damping.

The MIT command frame exposes k_p and k_d , so the position target becomes an impedance command. With fixed values, the policy can move the target position while the actuator controller provides the corresponding compliance. This raised the actuator bandwith by an order of magniture but it would not solve the previous issue where the policy was not having a still reward farming and not running. The limitation is that fixed gains impose the same compliance during all phases of the gait. A stiffness that is useful during stance is not necessarily appropriate during swing or at foot impact. The next step was therefore to make compliance part of the gait specification rather than treating it as a constant actuator setting. This approach was therefore not attempted further and no policy was trained for it.

4.6.4 A Fourier gait generator tuned by the policy in position and compliance space

ADD ARCHITECTURE SCREENSHOT

The main motivation for introducing a Fourier-based gait generator was to address a limitation observed with direct position control. In the latter approach, the policy was free to specify the position of each joint independently at every control step. This provided a large space in which the policy could exploit unintended behaviours, including repeated reward-gaming strategies observed during training. Rather than requiring the policy to discover the periodic structure of locomotion from scratch, the gait generator was therefore introduced to explicitly impose this structure while retaining enough freedom to adapt the gait. The policy no longer specifies every joint position independently, but instead controls a lower-dimensional Fourier representation of the gait. Once the compliance mode was unlocked, it was also natural to extend this parameterization to the stiffness and damping. Since the required compliance can reasonably be expected to depend on the phase of the gait, these parameters were represented using the same principle as the joint trajectories. The resulting generator therefore describes not only where the joints should move, but also how strongly they should track the generated trajectory throughout the gait cycle. This allows, for example, different compliance characteristics during impact and push-off.

To further reduce the dimensionality of the problem, a single generator was initially used for both legs. The right-leg trajectory was constrained to follow the same Fourier representation as the left leg, with a phase shift of π . This imposed the expected left/right alternation of a straight-line running gait while substantially reducing the number of parameters that the policy had to determine. This assumption, however, proved to be too restrictive. In particular, the learned policy tended to accept the reward penalty associated with continuously modifying the Fourier coefficients rather than maintaining a fixed set of coefficients. This behaviour suggests that the two legs could not be adequately described by a single phase-shifted trajectory and that independent corrections were important for maintaining the desired motion.

Despite this behaviour, the Fourier generator produced a working policy. Although the Fourier coefficients were updated continuously at each policy step, the resulting gait remained approximately periodic, with the generated trajectories varying by only a few percent over successive cycles. With all degrees of freedom released, the policy achieved stable running at speeds of up to approximately 3, m/s. This observation motivated a further experiment to determine whether

the learned gait itself was stable, independently of the policy’s continuous corrections. Static Fourier coefficients found to emulate the gait cycle that the policy was commanding. The robot was then commanded using only this fixed Fourier gait, without allowing the policy to modify the coefficients during execution. The robot was able to reach approximately the same running speed, but lost balance after only a few steps. The resulting behaviour demonstrates that the learned Fourier trajectory does not constitute a dynamically stable open-loop gait. Instead, the periodic trajectory provides the nominal locomotion pattern, while continuous policy updates are required to compensate for disturbances and state deviations and keep the robot upright.

4.6.5 Explicit oscillator against the Fourier series

The Fourier parameterisation is one member of a larger family of structured gait representations. Another canonical representation is an explicit limit-cycle oscillator, as used in the CPG literature [8, 3].

In the spirit of academic curiosity, the two representations were therefore compared experimentally. Sixteen cluster jobs shared the same plant, reward and curriculum schedule, with the generator being the main variable.

Building the oscillator arm required a foot-space formulation and a measured inverse-kinematics lookup table. Constructing this table exposed several properties of the mechanism that were not obvious from the CAD model. From the stance pose, both the cam and the thigh move the toe predominantly in the fore-aft direction. In addition, either sign of the cam motion shortens the nearly straight leg. The four-bar also contains a folded solution branch that must be filtered from the inverse-kinematics lookup table. The inversion has to preserve continuity between neighbouring cells; otherwise it can switch branches during the swing. Finally, the reachable vertical foot displacement at mid-stride is only approximately 8 cm because the leg is close to its straight configuration. The comparison is an overall tie. The oscillator appeared more sample-efficient from a cold start on the hardest tested milestone. At 133 M environment steps it reaches an episode length of 408 control steps, compared with 237 for the Fourier controller. The Fourier generator is more reliable across seeds. On the warm-started pitch-free plant it crosses the evaluation line in 5 of 5 runs at 2.45 m/s, compared with 3 of 5 runs at 2.40 m/s for the oscillator. The single best controller measured in this experiment was an oscillator run. It was the only one of the sixteen runs to execute the stop phase, reaching 2.9 m/s with a left/right asymmetry of 0.08. When run through the full curriculum ladder, the oscillator cleared m1 to m4, including a 100 m dash at m2 and 2.95 m/s at m3. The limitation appeared at m5, when roll was released. The oscillator reached an episode length of 337 control steps, compared with 3213 for the Fourier ladder at the matched budget. The plateau was confirmed after increasing the budget by a factor of 2.5. Both generators use the same three-parameter roll reflex, so the difference is attributed to the gait-generator part of the controller rather than the lateral reflex. The Fourier generator was nevertheless kept as it more thoroughly tested and better understood by the author. The result suggests that the structured action-space family is appropriate and thus the fourier formulation is not the only optimal one.

4.7 Curriculum design

The action-space experiments showed that providing structure to the gait was necessary, but it did not solve the balance problem by itself. The second major design variable was therefore the difficulty of the simulated plant.

4.7.1 The base-DOF locking ladder

The curriculum consists of six milestones, m1–m6. The base degrees of freedom are progressively released using equality constraints. The first milestone constrains the robot to a forward/backward rail, the following free the vertical dimension to let the policy learn to support the weight of the robot. Subsequent milestones release pitch and yaw before reaching the fully free plant.

The purpose is to postpone the hardest balance problem until a basic locomotion pattern has already been learned. On the first milestone the robot does not need to solve full-body balance. A ride-height rail and the measured inverse-kinematics lookup table make this simplified problem sufficiently constrained to be trainable.

Balance is therefore learned last rather than being required from the beginning. This is a form of curriculum over plant controllability rather than over the command or terrain as it was done in some papers.

4.7.2 Progression must be gated, not scheduled

The first versions of the curriculum used fixed, clock-driven progression. This caused the curriculum to release a new degree of freedom before the policy had learned the previous problem. In particular, the policy could be released before it had learned sufficient pitch balance, producing immediate falls. The progression was consequently changed to performance-gated advancement. The next milestone is unlocked only when the policy satisfies a performance criterion on the current one. The difficulty therefore follows the competence of the policy rather than wall-clock training time.

4.7.3 Exploration schedule

The exploration schedule produced another deadlock. The action standard deviation remained pinned at its maximum for approximately 150 M environment steps. The policy was therefore being trained under a level of exploration that prevented reliable exploitation of a useful gait.

The solution was to combine a forced annealing deadline with a performance gate. Exploration is allowed to remain high while the policy is improving, but it cannot remain indefinitely at the maximum value.

This change also closed the difference between stochastic and greedy evaluation observed in the earlier experiments.

4.7.4 Domain randomization from the measured plant

The domain randomisation ranges are inherited from the measurements in Section 3, including the mass-equivalent inertia uncertainty, timing jitter and actuation delay.

The purpose of domain randomization here is therefore robustness over a distribution of measured plants.

4.8 The pitch wall

The hardest curriculum transition occurred when pitch was released. Once pitch became a free degree of freedom, the policy repeatedly converged either to standing or slow creeping, or fell. No capture step emerged.

The plant itself cannot remain passively upright in the relevant configuration. A diagnostic of the learned policy showed that, while falling forward, the feet were placed approximately 8 cm behind

the centre of mass. The policy was therefore not solving the expected capture-step problem.

This failure could not be attributed to the policy alone. Scripted controllers were used to separate the plant limitation from the learning problem.

A zero-action controller combined with a re-solved stance keyframe survived for approximately 25 s. The required stance lean changed from approximately 5.7° to close to zero after the keyframe was re-solved. This showed that part of the original difficulty came from the reference stance itself.

Several modifications improved the duration of the learned behaviour without solving the wall. Re-solving the keyframe provided additional stability. A pitch-reflex prior also helped. An angular-momentum reward term produced another improvement, and rejuvenating the warm-started policy avoided some previously converged but unproductive states. None of these modifications produced sustained running.

The final resolution came by reverting the policy from a joystick controller to a "run as fast as possible" one. The earlier command-tracking objectives rewarded the policy for satisfying a motion target while remaining upright. Such an objective can be satisfied by standing or creeping when the commanded motion becomes difficult. The sprint objective instead makes standing unprofitable because progress is continuously rewarded and time is penalised.

With the sprint objective applied to the measured MIT-mode plant, one cold run produced the first true runner. The detailed performance and the margins of this controller are evaluated in [Section 5](#).

5 Results

This section is a recap of what this thesis achieved along with the result to characterise the deliverable.

5.1 Hardware delivered

Firstly, this thesis was quite involved in term of hardware and although the mechanical design was already done and provided, a lot of work was required to bring it from a state of expensive paper weight to a robot that could be controlled. The platform is a biped weighed at 15 kg. It carries four AKE90-8 QDD actuators (Table 3.1) and two AK60-39 for the hip roll. Its 13S2P pack holds 468 Wh and is rated 100 A continuous and 360 A for 2 s (Section 3.2.2). A Raspberry Pi 3B run the control loop at 200 Hz on a stock Linux. The platform can be backdriven and playback a recorded gait along with running the inference for a given policy. Low-level and policy control are executed on a Raspberry Pi 3B. On the stock Linux installation, the control stack sustains a 200,Hz control loop. The completed system supports actuator calibration, backdriving, recording and playback of joint trajectories, and execution of learned policies. A safety layer and black-box recorder were integrated into the control stack, and the principal operational functions are exposed through a web interface. Consequently, a new user can calibrate, enable, backdrive, replay trajectories, and run a policy without modifying or directly interacting with the control code.

Another important result of the hardware work is the experimental characterisation of the platform itself. The identification campaign established the quantities subsequently used to define and constrain the training plant, including actuator bandwidth under the tested control modes, control-loop timing and jitter, IMU noise, joint workspace, and actuator torque constants. These measurements also form part of the thesis results.

5.2 Running speed ceiling estimation

The physical running-speed ceiling of the platform was estimated independently of reinforcement learning in Section 3.6. The analysis places the maximum attainable speed at approximately 6,m/s.

This bound was obtained through two independent arguments. The first combines the available leg workspace with actuator torque and system latency to determine the achievable stance dynamics. The second considers the impulse required during stance to sustain the corresponding flight phase. Despite approaching the problem from different directions, the two estimates converge to a similar limiting speed, providing a useful consistency check on the result.

The estimated ceiling depends primarily on the usable leg workspace, the impulse that the actuators can deliver, and the distribution of mass within the leg. The associated sensitivity analysis further shows that actuator torque capability and distal leg mass are the dominant design levers, whereas increasing the supply voltage alone provides comparatively little improvement.

The 6,m/s estimate provides the reference and very optimistic upper ceiling against which the learned controller can be evaluated.

5.3 The runner

Using the measured plant model with MIT-mode actuation, a single cold training run with the sprint objective produced a controller that successfully completed the simulated 100,m dash in

all 16 greedy evaluation episodes. The mean completion time was 32.5s, corresponding to an average forward speed of 3.07m/s.

5.4 Legality of the gait against the measured envelope

The learned runner was subsequently audited against the experimentally measured hardware limits established in Section 3. This test replays greedy evaluation trajectories and checks the commanded motion at each control step against the measured actuator constraints, including torque, torque–speed capability, joint velocity, and thermal loading. The commanded torque remains within the permitted torque and torque–speed envelope for approximately 99.9% of the evaluated control steps. In this respect, the learned policy operates almost entirely inside the measured instantaneous actuation capability of the robot. The commanded current stay at the limit of the allowed thermal limits.

The audit nevertheless exposes one important violations. First, the swing-leg joint velocity reaches approximately twice the measured no-load actuator speed. Such motion is physically unattainable and therefore identifies a bug in the form of a missing velocity constraint in the simulation model that need to be patched.

5.5 Robustness margins

Robustness was assessed empirically using perturbed greedy evaluations. The tested perturbations included changes in robot mass, additional actuation delay, and external pushes.

The mass-scaling experiments indicate that the learned runner possesses appreciable control authority only when the effective plant mass is at or below the nominal training value. Performance degrades rapidly as the mass is increased above nominal, indicating limited force margin in the learned solution. Similarly, the policy was very sensitive to additional actuation delay than what is what trained against. This can likely be explained by the fact that error compound rapidly and while at a given time, the control action might have been stable, at $t+1$, the control action might already be much bigger. Since the policy was tasked with maximizing speed, it's using all its control authority for this task and have not more than what was needed in simulation to recover from disturbance.

This is the expected behaviour since the policy robustness is set at training by changing the noise distribution in the parameters it need to be robust against.

5.6 Deployment readiness

Several components required for hardware deployment were verified end-to-end. The trained neural-network policy can be exported to run on the raspberry pi CPU in less than 10ms. The deployment stack also includes the hardware safety governor and black-box logging system described above. At the platform level, the Raspberry Pi 3B sustains the 200Hz low-level control tick with the measured control workload. The complete learned runner was, however, not demonstrated as a 100m hardware sprint within the scope of the thesis. The principal computational limitation is policy inference on the Raspberry Pi 3B, whose execution time is slightly above the available per-step budget when combined with the remainder of the control stack. Two practical routes remain: upgrading the onboard computer, or retraining and validating the controller for a lower control frequency, such as 100Hz. Neither could be completed and experimentally validated before submission.

Accordingly, the final deliverable is an operational and experimentally characterised bipedal platform, together with a complete learning and deployment pipeline and a simulated running

controller that reaches 3.07,m/s.

6 Discussion

This chapter measures the distance between the objective introduced in Section 1.1 and the results reported in Section 5. It then discusses the lessons learned from the control methodology and ranks the changes that are most likely to close the remaining performance gap.

6.1 Method-level lessons

Several lessons emerged from the development of the learned controller.

Optimisers exploit objectives. The policy repeatedly found behaviours that maximised the numerical objective without satisfying the intended locomotion task. Examples included skating, penetrating the floor, and exploiting high-frequency contact dynamics.

This behaviour is an instance of Goodhart’s law. The limitation of encoding human intents in a cost function have been studied and this is an expected result [12].

The simulator must enforce the physical envelope. The final runner provides a second example of the same principle. Although the commanded torques remain almost entirely inside the measured torque–speed envelope, the swing-leg velocity reaches approximately twice the measured no-load actuator speed. The resulting gait therefore exploits a missing physical constraint in the simulator. This does not invalidate the learning framework, but it means that the achieved 3.07 m/s should be interpreted as an optimistic simulation result rather than a directly deployable hardware speed.

The implication for future training is that constraints which are physically absolute should preferably be encoded directly in the plant rather than only discouraged through the reward. In particular, actuator velocity limits and thermal behaviour should be included in the simulator before further optimisation of maximum speed.

The policy uses the available control authority. The perturbation experiments show that the final controller has little authority remaining above the nominal plant. Increasing the robot mass or introducing additional delay rapidly degrades performance. This behaviour is consistent with the optimisation objective: the policy was explicitly trained to maximise speed and therefore has an incentive to use the available actuator authority for propulsion rather than reserve it for disturbance rejection outside of its trained range.

Consequently, robustness should not be expected to emerge automatically. If robustness to mass uncertainty, latency, impacts, or other plant variations is required, those variations should be represented explicitly in the training distribution.

6.2 Mechanical limits and performance levers

The approximately 6 m/s physical ceiling derived in Section 3.6 provides a useful reference for deciding where further development effort should be spent. From the result of the sensitivity analysis, the following point can be made.

6.2.1 Increase the usable leg workspace

The first design objective should be to increase the usable fore–aft workspace of the foot. The available stroke directly enters the speed-ceiling calculation: for a given stride frequency, a larger usable foot sweep allows a larger stride and therefore a higher forward speed.

The four-bar geometry and segment dimensions should be optimised together to maximise the portion of the workspace that is useful during running along with the level of force anisotropy required.

6.2.2 Reduce the force anisotropy

A second and potentially inexpensive lever is to redesign the four-bar linkage so that more of the available actuator torque can be converted into fore-aft ground force.

At the current standing pose, the approximately 19:1 force anisotropy indicates that the mechanism is mechanically well suited to supporting vertical load but poorly suited to producing horizontal propulsion. Increasing motor torque would increase both capabilities, but would not address this missplacement of control authority that stem from the 4bar linkage anisotropy.

A redesigned linkage could therefore improve effective propulsive force without requiring a proportionally larger actuator. For this reason, linkage geometry should be explored before actuator scaling. The design problem should combine at least two objectives: maximising usable fore-aft workspace and maintaining favourable force transmission throughout the stance region.

6.2.3 Reduce distal leg mass

The sensitivity analysis also identifies moving leg mass as an important performance lever. Reducing the leg mass by 30 % increases the estimated speed ceiling by approximately 0.7 m/s.

The practical margin for improvement may, however, be limited because the existing distal links are already lightweight. Further reduction would therefore need to be achieved without sacrificing stiffness or durability.

Where reductions remain possible, priority should be given to distal mass rather than total robot mass. Mass located near the foot contributes strongly to the inertia that must be accelerated and reversed during each swing phase, whereas an equivalent mass reduction near the torso has a much smaller effect on swing dynamics.

6.2.4 Increase actuator torque density

If linkage optimisation and mass reduction are insufficient, increasing actuator capability is the next direct lever although a much pricier one.

For a future platform, actuator selection should consequently favour torque density and continuous thermal capability rather than simply the largest available peak-torque rating. Contemporary record-oriented humanoids appear to use substantially higher torque capability than DASH-01. The author guesstimate them to be around 400 Nm peak but no credible source could be found to confirm nor deny this value.

6.2.5 Bus voltage is not presently a priority

Increasing the bus voltage initially appears attractive because it raises the actuator no-load speed. The sensitivity study, however, indicates that a 30 % voltage increase produces essentially no increase in the current speed ceiling. At the operating point considered here, the mechanism is limited by torque, inertia, workspace, and available stance impulse before it reaches the no-load-speed limit. Increasing voltage should therefore not be treated as a primary performance lever for the present design. It may become relevant after other limitations have been removed, but the current results do not justify prioritising it.

7 Conclusion and future work

This thesis investigated how far reinforcement learning could push the running performance of DASH-01, a custom 15 kg biped, while grounding the controller in the measured limitations of the physical platform.

The initial ambition was to approach the bipedal 100 m speed record. The characterisation performed during this work showed that the current mechanism does not admit such performance: an independent first-principles analysis places its optimistic physical running-speed ceiling at approximately 6 m/s. The objective therefore evolved toward producing a demonstrator and, more importantly, a reusable methodology linking the mechanical design, plant identification, simulation, reinforcement learning and deployment.

7.1 What was delivered

Three principal outcomes were obtained.

First, DASH-01 was brought from a partially completed mechanical platform to an operational robot. The resulting system supports actuator and IMU calibration, backdriving, trajectory recording and playback, policy execution, workspace enforcement, emergency stopping and black-box logging. The robot can be operated through a web interface without requiring the user to modify the control software to run new policy. :contentReference[oaicite:1]index=1

Second, the robot was experimentally characterised to obtain a measured plant and to verify the CAD data. The identification established the joint workspace, actuator torque constants and bandwidth, control-loop latency and jitter, IMU noise characteristics and segment masses. These measurements were subsequently incorporated into the simulation and used to constrain both the learning problem and the analysis of the achievable running speed.

Third, a reinforcement-learning pipeline was developed from the measured plant to a deployable policy representation. Several action-space formulations and curricula were investigated. The final formulation combines a structured gait generator with impedance parameters, and produced a simulated runner completing the 100 m dash in 32.5 s, corresponding to an average speed of 3.07 m/s.

7.2 Limitations

The principal limitation is that the final running policy was not demonstrated on the physical robot. Although the deployment stack was completed and the Raspberry Pi 3B can sustain the low-level control workload, inference of the final policy together with the remainder of the software stack is slightly too slow for reliable operation at 200 Hz. Deployment therefore requires either more capable onboard compute or retraining and validation at a lower control frequency, such as 100 Hz.

The simulated running result must also be interpreted with care. The gait audit showed that the torque commands remain almost entirely inside the measured torque-speed envelope, but the swing-leg velocity reaches approximately twice the measured actuator no-load speed. This motion is physically unattainable and reveals a missing velocity constraint in the training plant that need to be patched. In addition, the commanded current operates close to the assumed continuous thermal limit, while the actuator thermal model was not experimentally identified.

More generally, the experiments showed that optimisation results cannot be interpreted from reward alone. Several policies exploited mismatches between the objective and the intended behaviour, including skating and non-physical contact behaviour. The final runner similarly

exploits a missing actuator-velocity constraint. A central methodological conclusion is therefore that reinforcement learning is only as physically meaningful as the plant model and objective against which it is optimised.

7.3 Future work

The next iteration of the project should begin with a morphology search informed by the measurements obtained in this thesis. The first priority is to redesign the four-bar linkage and leg dimensions jointly, with the objective of increasing the usable fore-aft workspace while reducing the strong force anisotropy of the current mechanism. This offers the possibility of obtaining more propulsive capability from the actuators already installed before increasing actuator size.

The second priority is to reduce distal leg mass where meaningful reductions remain possible. This should be considered together with linkage redesign, because changing segment lengths and transmission geometry also changes the distribution of moving mass and the resulting swing inertia.

If additional performance is still required after these changes, the actuator selection should then be revisited with an emphasis on greater torque density and continuous torque capability while staying in QDD territory. The measured sensitivity suggests that additional torque is the strongest remaining direct lever, but its benefit must be evaluated together with the mass and inertia penalty introduced by a larger drive.

Several changes are also required independently of maximum speed. The current point foot provides no direct contact moment and limits the available interaction with the ground. A carbon blade foot should therefore be evaluated. An impact or force sensor at the foot would additionally provide direct information about touchdown and ground reaction, rather than requiring the controller to infer contact indirectly from base motion.

Finally, the simulation should be updated before the next high-speed training campaign. Explicit actuator velocity limits and a validated thermal model should be incorporated so that future policies cannot exploit behaviours unavailable to the physical plant.

8 Appendix

8.1 From CAD to a simulable model

The procedure below is the one that worked, after the detours recorded in Section 3.3. The repository holds the scripts and the exported files.¹

1. **Re-assemble in one CAD system.** Each limb is exported as STEP with its origin centred on its joint, then assembled and exported to URDF with a plugin. Inertia tensors supplied from a second CAD system are validated against their reference frame explicitly rather than trusted, because that export arrived rotated.
2. **Accept that the URDF is an approximation.** URDF is a tree format, so the pushrod floats and the 4-bar is open. The exported URDF is kept for tooling that needs it, and it is never simulated.
3. **Close the loop in MJCF.** Two sites, one at the pushrod tip and one on the shin, are tied by `<equality><connect>`. The residual at the measured workspace poses is 10^{-14} .
4. **Replace CAD masses with weighed masses,** and rescale each body's inertia tensor by its own mass ratio. This removes the segment masses from the identification problem entirely.
5. **Re-settle the stance keyframe under gravity** after any plant change. A keyframe is an equilibrium of one particular ankle law, and re-settling is not idempotent: re-settling from an already crouched keyframe lands on the 4-bar's fold branch.

8.2 Component databases

Both databases are published as standalone tools.

Actuator review.² It holds 140 commercial integrated actuators, 121 motors, 121 gearboxes and 9 standalone drives, as four joined tables. A candidate can therefore be composed rather than only looked up. Modelling follows Lee et al., *How to Model Brushless Electric Motors for the Design of Lightweight Robotic Systems* (arXiv:2310.00080).

Battery review.³ It holds roughly 430 cell models, 54 BMS modules and 110 commercial packs, each row carrying a source URL. A synthesiser builds a pack from cell, BMS and series-parallel count. A synthesisable pack therefore plots on the same axes as a commercial one. Every current rating carries a duration stamp, and falls into a continuous, pulse-at- t or fault tier. The pulse tier interpolates as $I(t) = \sqrt{I_{\text{cont}}^2 + (I_{\text{pulse}}^2 - I_{\text{cont}}^2) t_{\text{pulse}}/t}$. Fields inferred rather than read off a datasheet are written as ESTIMATED, so that they still parse and can be filtered out.

¹https://github.com/nathmo/running_robot

²<https://github.com/nathmo/actuatorReview>

³<https://github.com/nathmo/batteryReview>

8.3 Logbook

Timeline reconstructed from the repository’s commit history, with the exception of the measurement campaign (Section 3.4) which was performed in parallel to the training. The logbook is a record of the work, not a tutorial. It is therefore selective and omits the many dead ends, the many failed experiments, and the many commits that were only a few lines of hyper-parameter tuning.

- 2026-04-13** Repository created. First framework: MuJoCo plus Stable-Baselines3 PPO on a generic legged environment.
- 2026-04-20** A dozen reward and hyper-parameter commits in one day. The robot learns to belly-flop, and the fall penalty, forward-speed weight, alive bonus and feet-air-time thresholds are each chased in turn.
- 2026-04-21** Tuning starts going backwards. Decision to attack sim-to-real separately on a toy problem.
- 2026-04-30** Pendulum environment, PPO, ONNX export and a Raspberry Pi runtime: the project’s first complete simulation-to-hardware transfer (Section 4.2).
- 2026-05-21** The generic-robot approach is abandoned. The DASH-01 URDF is exported from CAD and the simulator rewritten around it.
- 2026-05-22** Manual URDF repair: the closed loop of the parallel leg cannot be expressed in URDF, and the export angles are wrong.
- 2026-06-22** First training on the real geometry: a standing policy, converged two days later.
- 2026-07-06** **Anti-skating rebuild.** An instrumented rollout measures what the policy is doing: both feet grounded 57–66 % of steps, median swing one control step, -0.09 m/s of travel under a $+0.5$ m/s command. Three causes fixed at once: contact `condim 3→6`, the six motor masses welded onto their stator bodies ($5.73→12.83$ kg), and a standing keyframe settled under gravity.
- 2026-07-06** Hardware bring-up begins in parallel: motor test scripts, slow sine sweeps, torque-control safety limits.
- 2026-07-09** Workspace and joint-limit calibration on the real leg.
- 2026-07-10** Web interface to drive the robot: per-motor sliders, live leg visualisation, gait recording and playback.
- 2026-07-14** Naming settled to DASH-01. Two structural changes: the base-DOF locking curriculum and the Fourier gait controller.
- 2026-07-21** m3 falls in 1.5 s. Pitch reflex, angular-momentum reward and normalisation rejuvenation added.
- 2026-07-22** The real m3 failure is an entropy-gate deadlock: the action standard deviation sat pinned at maximum for a whole 150 M-step run.
- 2026-07-23** The decaying pitch assist turns into a crutch. The control rate moves from 50 to 200 Hz, with the reward made rate-invariant and every curriculum count rescaled.
- 2026-07-24** A diagnostic proves the m3 policy plants its feet 8 cm behind the centre of mass while falling forward: it never learned the capture step. An evaluator bug that evaluated at 100 m rather than the curriculum distance is found and fixed.
- 2026-07-28** **Gait-frequency bug.** The 200 Hz rescale had widened the learnable range to (0.5, 50) Hz, so a neutral action meant a 25 Hz gait clock. Remapped to (0.5, 4.0).

- 2026-07-29** The fix produces the best m3 controller so far. The residual 6 Hz footfall is traced, wrongly at first, to ankle-spring resonance.
- 2026-07-30** Generator A/B result: no clear winner. The full ladder is relaunched twice, once with the cadence fixes and once on the oscillator generator.
- 2026-08-04** **The measurement campaign opens.** Weighed masses replace the CAD placeholders, 12.83→15.14 kg. The toe's static force capability is mapped over the workspace: 96 % of reach, 19:1 anisotropy, the passive ankle 8× short.
- 2026-08-05** System identification debugged: two sign errors, per-joint refitting, reduced Jacobian damping. Peak joint torque corrected to what the gearbox delivers.
- 2026-08-06** **The ankle question closes by statics.** The real spring deflects 26.6° and drives the heel 11.9 mm through the floor at half body weight. The CAN feedback protocol is measured, and a warm-start bug is found: seed replicates had been silent duplicates.
- 2026-08-09** Commanded speeds are bounded by what the plant can reach. The walker gets the base-DOF ladder it never had.
- 2026-08-10** **The plant becomes the robot as rebuilt.** Per-joint motor velocity caps are enforced for the first time. The ankle becomes a rigid carbon tube and the drive bandwidth is bracketed. On the hardware side, a failure that cost a joint prompts the three-tier flight recorder and the pre-move guard.
- 2026-08-11** First full ladder on the rebuilt plant: twelve jobs, about 23 h wall clock. m2 walks out the full 60 s cap; m5 is a hard wall. A scripted, no-learning walk controller is written as a baseline.
- 2026-08-19** **Foot-shape study: sixteen runs, negative result.** Reaching the answer took three wrong readings of the same data, and the harness's own rule, that a per-arm difference smaller than the within-rung spread is not a result, killed all of them.
- 2026-08-25** **Foot arc geometry measured.** Forward reach 31 cm, backward reach 2.7 cm. Re-solving the balanced keyframe takes the scripted controller at m3 from 0.91 s to 24.7 s.

References

- [1] Max A. Aebi. Dash-01: A High Speed Bipedal Running Robot. From Kinematic Design to Mechanical Implementation. Master’s thesis, EPFL, Lausanne, Switzerland, August 2026. Advisers: Mohamed Bouri, Alain Preneloup.
- [2] Associated Press. Chinese humanoid robot breaks speed record at world games. AP News, 2026. <https://apnews.com/article/chinese-humanoid-robot-beijing-world-games-record-069b76ee6748154fd67bec39fa927cf6>.
- [3] Guillaume Bellegarda and Auke Ijspeert. CPG-RL: Learning central pattern generators for quadruped locomotion. *IEEE Robotics and Automation Letters*, 2022. arXiv:2211.00458.
- [4] Miroslav Bogdanovic, Majid Khadiv, and Ludovic Righetti. Learning variable impedance control for contact sensitive tasks. *IEEE Robotics and Automation Letters*, 5(4):6129–6136, 2020. arXiv:1907.07500.
- [5] Devin Crowley, Jeremy Dao, Helei Duan, Kevin Green, Jonathan Hurst, and Alan Fern. Optimizing bipedal locomotion for the 100m dash with comparison to human running. *arXiv preprint*, 2025. arXiv:2508.03070.
- [6] Nicolas Heess, Dhruva TB, Srinivasan Sriram, Jay Lemmon, Josh Merel, Greg Wayne, Yuval Tassa, Tom Erez, Ziyu Wang, S. M. Ali Eslami, Martin Riedmiller, and David Silver. Emergence of locomotion behaviours in rich environments. *arXiv preprint*, 2017. arXiv:1707.02286.
- [7] Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, Dario Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26):eaau5872, 2019.
- [8] Auke Jan Ijspeert. Central pattern generators for locomotion control in animals and robots: a review. *Neural Networks*, 21(4):642–653, 2008.
- [9] Atil Iscen, Ken Caluwaerts, Jie Tan, Tingnan Zhang, Erwin Coumans, Vikas Sindhwani, and Vincent Vanhoucke. Policies modulating trajectory generators. *Conference on Robot Learning (CoRL)*, 2018. arXiv:1910.02812.
- [10] Benjamin Katz, Jared Di Carlo, and Sangbae Kim. Mini Cheetah: A platform for pushing the limits of dynamic quadruped control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 6295–6301, 2019.
- [11] Scott Kuindersma, Robin Deits, Maurice Fallon, Andrés Valenzuela, Hongkai Dai, Frank Permenter, Twan Koolen, Pat Marion, and Russ Tedrake. Optimization-based locomotion planning, estimation, and control design for the Atlas humanoid robot. *Autonomous Robots*, 40(3):429–455, 2016.
- [12] Antoine Maier, Aude Maier, and Tom David. Take Goodhart seriously: Principled limit on General-Purpose AI optimization. *arXiv preprint*, 2025. arXiv:2510.02840.
- [13] Nathann Morand. actuatorReview. GitHub repository, 2026. Accessed 2026-08-30.
- [14] Nathann Morand. Motormap: An Interactive Actuator Explorer. Streamlit web application, 2026. Deployed instance of the actuatorReview repository, accessed 2026-08-30.
- [15] Nathann Morand. running_robot. GitHub repository, 2026. Workspace calibration, web UI and dynamic-identification pipeline; accessed 2026-09-01.
- [16] OpenAI, Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. Solving Rubik’s Cube with a robot hand. *arXiv preprint*, 2019. arXiv:1910.07113.
- [17] Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne. DeepMimic: Example-guided deep reinforcement learning of physics-based character skills. *ACM Transactions on Graphics (SIGGRAPH)*, 37(4), 2018.

- [18] Nuofan Qiu, Chaoyang Song, and Fang Wan. Describing robots from design to learning: Towards an interactive lifecycle representation of robots. In *2024 International Conference on Advanced Robotics and Mechatronics (ICARM)*, pages 1081–1086, 2024.
- [19] Marc Raibert, Kevin Blankespoor, Gabriel Nelson, and Rob Playter. BigDog, the rough-terrain quadruped robot. In *IFAC Proceedings Volumes (17th IFAC World Congress)*, volume 41, pages 10822–10825, 2008.
- [20] Marc H. Raibert. *Legged Robots That Balance*. MIT Press, Cambridge, MA, 1986.
- [21] Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Conference on Robot Learning (CoRL)*, pages 91–100, 2021.
- [22] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint*, 2017. arXiv:1707.06347.
- [23] Nima Shafii, Siavash Aslani, Omid Mohamad Nezami, and Saeed Shiry. Evolution of biped walking using truncated Fourier series and particle swarm optimization. In *RoboCup 2009: Robot Soccer World Cup XIII*, volume 5949 of *Lecture Notes in Computer Science*, pages 344–354. Springer, 2010.
- [24] Jonah Siekmann, Yesh Godse, Alan Fern, and Jonathan Hurst. Sim-to-real learning of all common bipedal gaits via periodic reward composition. *IEEE International Conference on Robotics and Automation (ICRA)*, 2021. arXiv:2011.01387.
- [25] Jie Tan, Tingnan Zhang, Erwin Coumans, Atıl İscen, Yunfei Bai, Danijar Hafner, Steven Bohez, and Vincent Vanhoucke. Sim-to-real: Learning agile locomotion for quadruped robots. In *Robotics: Science and Systems (RSS)*, 2018. arXiv:1804.10332.
- [26] Waseda University. Robots of Waseda: WABOT-1. YouTube, May 2026. YouTube video, published May 7, 2026. <https://www.youtube.com/watch?v=n0oL1sHAKwE>.
- [27] WhatIMakeOfThis. Asimo Walking. YouTube, July 2020. YouTube video, published July 16, 2020. <https://www.youtube.com/watch?v=fAFACFIWDA8>.
- [28] Zhaoming Xie, Patrick Clary, Jeremy Dao, Pedro Morais, Jonathan Hurst, and Michiel van de Panne. Learning locomotion skills for Cassie: Iterative design and sim-to-real. In *Conference on Robot Learning (CoRL)*, pages 317–329, 2019.